

Docs tip: All developer.woocommerce.com/docs/ pages have a **"Copy page content as markdown"** button in the top-right that copies the full page as clean Markdown to clipboard — perfect for feeding to AI agents.

```
// Cart/Checkout
'woocommerce_add_to_cart_validation' // Validate before add to cart
'woocommerce_add_cart_item_data'    // Add custom cart item data
'woocommerce_get_item_data'         // Display custom cart item data
'woocommerce_checkout_fields'       // Modify checkout fields
'woocommerce_billing_fields'         // Modify billing fields only
'woocommerce_shipping_fields'       // Modify shipping fields only

// Orders
'woocommerce_order_number'          // Custom order numbering
'woocommerce_payment_complete_order_status' // Status after payment

// Shipping
'woocommerce_shipping_methods'      // Register shipping methods
'woocommerce_package_rates'         // Modify shipping rates
```

1.4 Custom Product Types

```
// Register a custom product type
function register_custom_product_type() {
    class WC_Product_Custom extends WC_Product {
        public function __construct($product) {
            $this->product_type = 'custom';
            parent::__construct($product);
        }

        public function get_type() {
            return 'custom';
        }
    }
}
add_action('init', 'register_custom_product_type');

// Add to product type selector
add_filter('product_type_selector', function($types) {
    $types['custom'] = __('Custom Product', 'my-plugin');
    return $types;
});

// Custom product data tabs
add_filter('woocommerce_product_data_tabs', function($tabs) {
    $tabs['custom'] = [
        'label' => __('Custom Data', 'my-plugin'),
        'target' => 'custom_product_data',
        'class' => ['show_if_custom'],
        'priority' => 21,
    ];
    return $tabs;
});
```

1.5 Custom Payment Gateways

```
class WC_Gateway_Custom extends WC_Payment_Gateway {
    public function __construct() {
        $this->id = 'custom_gateway';
        $this->icon = '';
        $this->has_fields = true;
        $this->method_title = __('Custom Gateway', 'my-plugin');
        $this->method_description = __('Custom payment method description', 'my-plugin');

        $this->init_form_fields();
        $this->init_settings();

        $this->title = $this->get_option('title');

        add_action('woocommerce_update_options_payment_gateways_' . $this->id, [$this, 'process_admin_options']);
    }

    public function init_form_fields() {
        $this->form_fields = [
            'enabled' => [
                'title' => __('Enable/Disable', 'my-plugin'),
                'type' => 'checkbox',
                'label' => __('Enable Custom Gateway', 'my-plugin'),
                'default' => 'no',
            ],
            'title' => [
                'title' => __('Title', 'my-plugin'),
                'type' => 'text',
                'description' => __('Payment method title', 'my-plugin'),
                'default' => __('Custom Payment', 'my-plugin'),
            ],
        ];
    }

    public function process_payment($order_id) {
        $order = wc_get_order($order_id);
        // Process payment logic here
        $order->payment_complete();
        $order->reduce_order_stock();

        return [
            'result' => 'success',
            'redirect' => $this->get_return_url($order),
        ];
    }
}

add_filter('woocommerce_payment_gateways', function($gateways) {
    $gateways[] = 'WC_Gateway_Custom';
    return $gateways;
});
```

1.6 Custom Shipping Methods

```
class WC_Shipping_Custom extends WC_Shipping_Method {
    public function __construct($instance_id = 0) {
        $this->id = 'custom_shipping';
        $this->instance_id = absint($instance_id);
        $this->method_title = __('Custom Shipping', 'my-plugin');
        $this->method_description = __('Custom shipping calculation', 'my-plugin');
        $this->supports = ['shipping-zones', 'instance-settings'];

        $this->init();
    }

    public function init() {
        $this->init_form_fields();
        $this->init_settings();
        $this->title = $this->get_option('title');
    }

    public function calculate_shipping($package = []) {
        $rate = [
            'id' => $this->get_rate_id(),
            'label' => $this->title,
        ];
    }
}
```

```

        'cost' => '10.00',
        'calc_tax' => 'per_item',
    ];
    $this->add_rate($rate);
}
}

add_filter('woocommerce_shipping_methods', function($methods) {
    $methods['custom_shipping'] = 'WC_Shipping_Custom';
    return $methods;
});

```

1.7 Template Structure & Overrides

Template override priority chain: 1. `{child-theme}/woocommerce/{template}` 2. `{parent-theme}/woocommerce/{template}` 3. `wp-content/plugins/woocommerce/templates/{template}`

Key templates to know:

```

templates/
├── single-product/
│   ├── product-image.php      # Featured image
│   ├── product-thumbnails.php # Gallery thumbnails
│   ├── price.php              # Price display
│   ├── add-to-cart.php        # Add to cart form
│   ├── rating.php             # Star rating
│   ├── meta.php               # SKU/categories/tags
│   ├── tabs/tabs.php          # Product tabs wrapper
│   └── related.php            # Related products
├── cart/
│   ├── cart.php               # Cart page
│   ├── cart-totals.php        # Cart totals
│   ├── cart-empty.php        # Empty cart message
│   └── mini-cart.php          # Mini cart widget
├── checkout/
│   ├── form-checkout.php      # Full checkout form
│   ├── form-billing.php       # Billing fields
│   ├── form-shipping.php      # Shipping fields
│   ├── form-coupon.php       # Coupon form
│   ├── form-login.php        # Login form
│   ├── payment.php            # Payment methods
│   └── thankyou.php           # Order received
├── myaccount/
│   ├── my-account.php         # Account dashboard
│   ├── form-login.php         # Login form
│   ├── orders.php             # Order history
│   ├── dashboard.php          # Dashboard
│   └── form-edit-account.php   # Edit account
├── loop/
│   ├── add-to-cart.php        # Loop add to cart button
│   ├── price.php              # Loop price
│   ├── sale-flash.php         # Sale badge
│   ├── rating.php             # Loop rating
│   └── no-products-found.php   # Empty results
└── emails/
    ├── admin-new-order.php     # Admin new order notification
    ├── customer-completed-order.php
    ├── customer-processing-order.php
    ├── customer-on-hold-order.php
    └── email-header.php / email-footer.php

```

1.8 HPOS — High-Performance Order Storage

Status: Stable since WooCommerce 8.2 (October 2023). **Enabled by default** on new installations. Fully WP 7.0 compatible.

What HPOS does: - Moves order data from `wp_posts` + `wp_postmeta` to dedicated `wp_wc_orders` + `wp_wc_order_meta` tables - Dramatically improves order query performance (no more postmeta JOINS) - Supports data synchronization mode for backward compatibility

Enabling HPOS:

```

# Via WP-CLI
wp wc cot enable      # Enable HPOS only
wp wc cot enable --sync # Enable with sync (legacy + HPOS tables)
wp wc cot migrate     # Migrate existing orders
wp wc cot count       # Count orders in each table
wp wc cot verify      # Verify all orders migrated

```

HPOS Compatibility for Extensions:

```

// Declare HPOS compatibility in plugin header
/**
 * Plugin Name: My WooCommerce Extension
 * ...
 * WC requires at least: 8.0
 * WC tested up to: 11.1
 * HPOS compatibility: true
 */

// Use WC_Order_Query instead of WP_Query for orders
$order = wc_get_orders([
    'limit' => 10,
    'status' => ['completed', 'processing'],
    'date_created' => '>2026-01-01',
    'customer_id' => 42,
    'return' => 'objects',
]);

// Never do this with HPOS:
// $order = new WP_Query(['post_type' => 'shop_order', ...]); // ■ BROKEN with HPOS

// Always use WC CRUD:
$order = wc_get_order($order_id); // ■ Works with both
$order->get_billing_email();
$order->get_total();
$order->update_status('completed');

```

HPOS Compatibility Scanner: Use the [HPOS Compatibility Scanner plugin](#) to scan your codebase for incompatible direct database access patterns.

1.9 WooCommerce REST API

Base URL: `https://yoursite.com/wp-json/wc/v3/`

Authentication: API keys (Consumer Key + Consumer Secret) via WooCommerce > Settings > Advanced > REST API. Or Application Passwords (WP 5.6+).

Key Endpoints:

GET	/products	# List products
POST	/products	# Create product
GET	/products/{id}	# Get product
PUT	/products/{id}	# Update product
DELETE	/products/{id}	# Delete product
POST	/products/batch	# Batch create/update/delete
GET	/orders	# List orders
POST	/orders	# Create order
GET	/orders/{id}	# Get order
PUT	/orders/{id}	# Update order
DELETE	/orders/{id}	# Delete order
GET	/customers	# List customers
POST	/customers	# Create customer
GET	/customers/{id}	# Get customer
GET	/products/categories	# Product categories
GET	/products/tags	# Product tags
GET	/products/attributes	# Product attributes
POST	/products/reviews	# Create product review
GET	/coupons	# List coupons
POST	/coupons	# Create coupon
GET	/reports	# Report endpoints
GET	/reports/sales	# Sales report
GET	/reports/products	# Product report
GET	/reports/orders/totals	# Order totals
GET	/shipping/zones	# Shipping zones
GET	/payment_gateways	# Available payment methods
GET	/system_status	# System status (needs admin privs)

REST API + AI Agent Pattern:

```
// Create a custom REST endpoint for AI agent access
add_action('rest_api_init', function() {
    register_rest_route('ai-woocommerce/v1', '/product-description', [
        'methods' => 'POST',
        'callback' => 'generate_product_description',
        'permission_callback' => function() {
            return current_user_can('edit_products');
        },
        'args' => [
            'product_id' => ['required' => true, 'type' => 'integer'],
            'keywords' => ['type' => 'array'],
            'tone' => ['type' => 'string', 'default' => 'professional'],
        ],
    ]);
});

// Or use WP 7.0 AI Client in the callback
function generate_product_description($request) {
    $product = wc_get_product($request['product_id']);
    $product_name = $product->get_name();

    $prompt = sprintf(
        'Write a compelling WooCommerce product description for "%s". Keywords: %s. Tone: %s.',
        $product_name,
        implode(' ', $request['keywords'] ?? []),
        $request['tone']
    );

    $description = wp_ai_client_prompt($prompt)
        ->using_temperature(0.8)
        ->generate_text();

    return ['success' => true, 'description' => $description];
}
```

1.10 WooCommerce + ACF Integration

Essential ACF Pro features for WooCommerce: - **Repeater Fields** — product specifications, FAQ sections, size charts - **Flexible Content** — dynamic product page sections - **ACF Blocks** — custom WooCommerce blocks - **Options Pages** — global store settings

```
// Add ACF fields to WooCommerce products
add_action('acf/init', function() {
    acf_add_local_field_group([
        'key' => 'group_product_specs',
        'title' => 'Product Specifications',
        'location' => [
            [
                [
                    'param' => 'post_type',
                    'operator' => '==',
                    'value' => 'product',
                ],
            ],
        ],
        'fields' => [
            [
                'key' => 'field_product_dimensions',
                'label' => 'Dimensions',
                'name' => 'product_dimensions',
                'type' => 'text',
            ],
            [
                'key' => 'field_product_material',
                'label' => 'Material',
                'name' => 'product_material',
                'type' => 'select',
                'choices' => [
                    'cotton' => 'Cotton',
                    'polyester' => 'Polyester',
                    'wool' => 'Wool',
                ],
            ],
        ],
    ]);
});

// Display ACF fields in product template
add_action('woocommerce_single_product_summary', function() {
    $dimensions = get_field('product_dimensions');
    $material = get_field('product_material');
    if ($dimensions || $material) {
        echo '<div class="product-specifications">';
        if ($dimensions) echo '<p><strong>Dimensions:</strong> ' . esc_html($dimensions) . '</p>';
        if ($material) echo '<p><strong>Material:</strong> ' . esc_html($material) . '</p>';
        echo '</div>';
    }
}, 25);
```

ACF + ACF MCP Server: Use [acf-mcp-server](#) (TypeScript MCP) for AI-assisted ACF field group management with **dryRun** + auto-backup. Preferred over Novamira PHP for ACF work.

2. AI + WooCommerce Integration

2.1 MCP Servers for WooCommerce

Repo	Stars	Description	Status
xmichal88x/woocommerce-mcp-server	■0	New WooCommerce MCP server — direct WooCommerce operations via MCP	Fresh (July 2026)
multideskio/agentpress	■0	MCP server for WordPress with direct DB access. Agents access posts, products, orders, and any DB table with granular permissions	Fresh (July 2026)
Marksab001/Agent2Wp	■0	Connect AI agents to WordPress via MCP. Execute PHP, WP-CLI, manage files directly	Fresh (July 2026)
themewireco/synchronity	—	MCP server for WP synchronization	July 2026
zaherg/wp-blockmarkup-mcp-server	—	Block markup MCP integration for WP	July 2026

WooCommerce now has official MCP integration! The developer docs (<https://developer.woocommerce.com/docs/features/mcp-integration/>) document a built-in MCP layer for WooCommerce operations — the first official eCommerce MCP in the WordPress ecosystem.

2.2 AI Plugins for WooCommerce (The Best Options)

Plugin	Type	Price	What It Does
WooCommerce AI Product Advisor	Official Woo	Free	AI co-pilot reviews your catalog, recommends improvements, one-click apply. Product listing optimization
CopyCraft	OS	Free	AI-powered product descriptions using GPT
UCP Checkout for WooCommerce	OS	Free	Enable AI agents (ChatGPT/Gemini/Claude) to discover and purchase products via Universal Commerce Protocol
Helpmate	OS	Free	AI chatbot with 24/7 answers, product suggestions, smart coupons, order tracking
Woo Smart Chatbot	OS	Free	GPT-powered product recommendation chatbot
AI Puffer	Freemium	Free/Premium	30K+ installs. AI automation engine + voice agents + vector DB. Content + SEO + chat
AI Bud	Freemium	Free/Premium	Model fine-tuning for WooCommerce. Custom AI models for your store
Dominopost	Freemium	Free/Premium	Full article structure: auto ToC, CTAs, key takeaways. Blog + product content

2.3 n8n WooCommerce Automation Workflows

Pre-built WooCommerce n8n nodes: - WooCommerce Trigger (new order, order status change, new customer) - WooCommerce Action (create/update product, create order, update customer) - Webhook → Telegram notifications for new orders

Example: Order Notification to Telegram

```
WooCommerce Trigger (New Order)
→ Format data (extract: order_id, total, customer_name, items)
→ Telegram Send Message (formatted order summary)
```

Example: AI-Powered Product Upload Pipeline

```
Google Sheets Trigger (new row with product data)
→ OpenAI (generate SEO-optimized description from specs)
→ Notion (store draft for approval)
→ WooCommerce Create Product (publish after approval)
```

2.4 WooCommerce + WP 7.0 AI Client

Provider-agnostic AI in WooCommerce:

```
// Generate product descriptions from product data
function generate_ai_product_description($product) {
    $name       = $product->get_name();
    $categories = wp_get_post_terms($product->get_id(), 'product_cat', ['fields' => 'names']);
    $attributes = $product->get_attributes();

    $prompt = sprintf(
        'Write a compelling eCommerce product description for "%s" in category: %s. Features: %s. Include SEO keywords naturally. 150-200 words.',
        $name,
        implode(' ', $categories),
        json_encode($attributes)
    );

    $description = wp_ai_client_prompt($prompt)
        ->using_temperature(0.8)
        ->using_model_preference('claude-sonnet-4-6', 'gemini-3.1-pro-preview', 'gpt-5.4')
        ->generate_text();

    return $description;
}

// Generate alt text for product images
function generate_product_alt_text($attachment_id, $product_id) {
    $product = wc_get_product($product_id);
    $image_desc = get_the_title($attachment_id);

    $alt = wp_ai_client_prompt(
        "Generate SEO-optimized alt text for a product image: $product->get_name() - $image_desc. Max 125 chars."
    )->generate_text();

    update_post_meta($attachment_id, '_wp_attachment_image_alt', $alt);
}

// AI-powered product categorization
function ai_suggest_categories($product_name, $description) {
    $categories = get_terms(['taxonomy' => 'product_cat', 'hide_empty' => false]);
    $cat_names  = wp_list_pluck($categories, 'name');

    $suggestion = wp_ai_client_prompt(
        "Given this product: {$product_name} - {$description}. Which of these categories fits best? ". implode(' ', $cat_names) . ". Return only the category name."
    );
}
```

```
)->using_temperature(0.3)->generate_text();  
    return $suggestion;  
}
```

2.5 AI Agent Patterns for WooCommerce

Pattern 1: Product Catalog Audit Agent

- 1. Fetch all products via REST API GET /wc/v3/products
- 2. For each product, check: missing description, low-res images, no categories, no tags
- 3. Generate AI improvement suggestions
- 4. Apply fixes via REST API batch update

Pattern 2: Dynamic Pricing Agent

- 1. Monitor competitor prices (scraper → webhook)
- 2. Compare with store prices via REST API
- 3. AI decides price adjustments
- 4. Apply bulk price updates

Pattern 3: Order Fulfillment Agent

- 1. Poll for new orders via REST API
- 2. Validate inventory
- 3. Generate packing slips (PDF)
- 4. Send shipping notifications
- 5. Update tracking numbers

3. Divi 5 + WooCommerce

3.1 Divi 5 WooCommerce Modules (Complete Reference)

Divi 5 ships **25 native WooCommerce modules** (all render-confirmed in builder v5.9.0, divi5-skill v0.6.3). All are self-closing blocks.

Showcase / Collection Modules

Slug	Module	Key Settings
divi/shop	Woo Products Grid	type: recent/featured/sale/best_selling/top_rated/product_category. postsNumber, columnsNumber, orderby, useCurrentLoop
divi/woocommerce-related-products	Related Products	Posts number, columns, orderby. Uses product categories
divi/woocommerce-cross-sells	Cross-Sells	Cart cross-sell items
divi/woocommerce-product-upsell	Up-Sells	Product up-sell items

Single-Product Element Modules

Slug	Module	Notes
divi/woocommerce-product-title	Product Title	Heading level configurable
divi/woocommerce-product-price	Product Price	Shows regular + sale price
divi/woocommerce-product-images	Product Images	Main image + sale badge
divi/woocommerce-product-gallery	Product Gallery	Grid of product images
divi/woocommerce-product-description	Description	Full or short description
divi/woocommerce-product-add-to-cart	Add to Cart	Qty selector + button. Toggle stock display
divi/woocommerce-product-meta	Product Meta	SKU, categories, tags
divi/woocommerce-product-rating	Star Rating	Stars display
divi/woocommerce-product-reviews	Reviews	Review list + review form
divi/woocommerce-product-stock	Stock Status	"X in stock" display
divi/woocommerce-product-tabs	Product Tabs	Description/Additional info/Reviews tabs
divi/woocommerce-product-additional-info	Additional Info	Attributes table
divi/woocommerce-breadcrumb	Breadcrumb	Woo breadcrumb trail

Cart & Checkout Modules

Slug	Module	Notes
divi/woocommerce-cart-products	Cart Products	Line items table
divi/woocommerce-cart-totals	Cart Totals	Subtotal/total + checkout button
divi/woocommerce-cart-notice	Cart Notices	"Added to cart" messages
divi/woocommerce-checkout-billing	Billing Fields	Full billing form
divi/woocommerce-checkout-shipping	Shipping Fields	Ship to different address
divi/woocommerce-checkout-additional-info	Additional Info	Order notes textarea
divi/woocommerce-checkout-order-details	Order Details	Order review table
divi/woocommerce-checkout-payment-info	Payment Info	Payment methods

3.2 Divi 5 JSON Patterns for WooCommerce

Product Page Template (Theme Builder):

```
{  
  "type": "et_pb_template",  
  "template_type": "product",  
  "elements": [  
    {  
      "type": "et_pb_section",  
      "elements": [  
        {  
          "type": "et_pb_row",  
          "elements": [  
            {  
              "type": "et_pb_column",  
              "elements": [  

```

```
{
  "type": "divi/woocommerce-product-images",
  "builderVersion": "5.9.0",
  "content": {"advanced": {"product": {"desktop": {"value": "dynamic"}}}}
},
{
  "type": "et_pb_column",
  "elements": [
    {
      "type": "divi/woocommerce-breadcrumb",
      "builderVersion": "5.9.0"
    },
    {
      "type": "divi/woocommerce-product-title",
      "builderVersion": "5.9.0",
      "content": {"advanced": {"product": {"desktop": {"value": "dynamic"}}}},
      "title": {"decoration": {"font": {"font": {"desktop": {"value": {"headingLevel": "h1"}}}}}
    },
    {
      "type": "divi/woocommerce-product-price",
      "builderVersion": "5.9.0",
      "content": {"advanced": {"product": {"desktop": {"value": "dynamic"}}}}
    },
    {
      "type": "divi/woocommerce-product-description",
      "builderVersion": "5.9.0",
      "content": {"advanced": {"product": {"desktop": {"value": "dynamic"}}}}
    },
    {
      "type": "divi/woocommerce-product-add-to-cart",
      "builderVersion": "5.9.0",
      "content": {"advanced": {"product": {"desktop": {"value": "dynamic"}}}},
      "elements": {"advanced": {"showQuantity": {"desktop": {"value": "on"}}}}
    },
    {
      "type": "divi/woocommerce-product-meta",
      "builderVersion": "5.9.0",
      "content": {"advanced": {"product": {"desktop": {"value": "dynamic"}}}}
    }
  ]
}
}
```

Shop Archive with Loop Builder: - Use `divi/shop` with `type: "recent"` and `columnsNumber: "3"` for the main product grid - For custom loop layouts, combine Loop Builder with WooCommerce single-product element modules - Theme Builder templates: Product Archive → `divi/shop` with `useCurrentLoop: "on"`

3.3 Divi 5 + WooCommerce: MCP & AI Tools

IA Webmaster Bridge (🔱1): 108 MCP tools for Divi 5. Can build complete WooCommerce shops from prompts. Includes Theme Builder, design system, 8-layer security. [GitHub](#)

divi5-skill (🔱16): 15 modular `.md` files including `DIVI5-MODULES-WOOCOMMERCE.md`. Teaches AI to write Divi 5 WooCommerce JSON directly. No MCP needed. [GitHub](#)

DiviOps (🔱15): MCP server + WP plugin for direct Divi 5 module manipulation. [GitHub](#)

3.4 Divi 5 WooCommerce Pitfalls

Cart/Checkout modules fatal on REST insert when cart is empty. Boot a WC cart first: `wc_load_cart(); WC()->cart->add_to_cart($id);` before inserting checkout modules programmatically. In normal Theme Builder use this is a non-issue.

Single-product modules need "dynamic" or product ID in `content.advanced.product`. Without it, modules render empty.

Gallery module needs actual gallery images. If product has only one image, `product-images` (main image) works; `product-gallery` needs multiple images.

AI defaults to HTML dump. Prompt AI: "Use separate WooCommerce Divi modules — shop/products grid, single-product elements (title, price, images, add-to-cart), cart, checkout."

Squad Modules Lite (65 free Divi 5 native modules) is the best reference implementation for custom WooCommerce module architecture in Divi 5. [GitHub](#)

4. Plugins & Tools Ecosystem

4.1 Essential WooCommerce Plugins (Free)

Plugin	WP.org Link	Purpose
WooCommerce	wordpress.org/plugins/woocommerce/	Core eCommerce
WooCommerce Stripe Gateway	wordpress.org/plugins/woocommerce-gateway-stripe/	Payments
WooCommerce PayPal Payments	wordpress.org/plugins/woocommerce-paypal-payments/	PayPal + Venmo
WooCommerce Shipping & Tax	wordpress.org/plugins/woocommerce-services/	Live rates + label printing
WooCommerce Subscriptions	woocommerce.com/products/woocommerce-subscriptions/	Recurring payments
WooCommerce Bookings	woocommerce.com/products/woocommerce-bookings/	Appointment booking
Mailchimp for WooCommerce	wordpress.org/plugins/mailchimp-for-woocommerce/	Email marketing
Yoast SEO for WooCommerce	wordpress.org/plugins/yoast-seo-woocommerce/	SEO optimization
WooCommerce Multilingual	wordpress.org/plugins/woocommerce-multilingual/	WPML integration
WooCommerce Customizer	wordpress.org/plugins/woocommerce-customizer/	Button text/checkout customization

4.2 Essential WooCommerce Plugins (Premium)

Plugin	Price	Best For
WooCommerce Subscriptions	\$199/yr	Recurring/membership stores
WooCommerce Bookings	\$249/yr	Service/appointment businesses
WooCommerce Memberships	\$199/yr	Content + product gating
WooCommerce Product Add-Ons	\$49/yr	Customizable products
WooCommerce Dynamic Pricing	\$129/yr	Bulk/role-based pricing
WooCommerce Table Rate Shipping	\$99/yr	Complex shipping rules

WooCommerce Product Bundles	\$49/yr	Kit/bundle products
-----------------------------	---------	---------------------

4.3 AI WooCommerce Plugins — Comparison

Plugin	Free Tier	AI Features	WooCommerce-Specific	WP 7.0 Compatible
WooCommerce AI Product Advisor	■ Free	Product listing optimization, catalog audit	■ 100% WooCommerce	■
AI Puffer	■ Free	Content, SEO, voice agents, vector DB	General WP + Woo	■ (30K+ installs)
AI Bud	■ Free	Model fine-tuning	General WP	■
Dominopost	■ Free	Full article structure (ToC, CTAs)	Content only	■
CopyCraft	■ Free OS	GPT product descriptions	■ WooCommerce	■
Helpmate	■ Free OS	AI chatbot + product suggestions	■ WooCommerce	■

4.4 Free vs Paid — When to Pay

Need	Free Solution	Paid Upgrade Trigger
Product descriptions	CopyCraft + WP AI Client	Bulk AI description overhaul
AI Chatbot	Helpmate / Woo Smart Chatbot	Need custom training
SEO	Yoast SEO for WooCommerce	RankMath Pro if heavy schema
Automation	n8n (self-host) + WooCommerce node	n8n Cloud for scale
Product recommendations	WooCommerce Related Products (native)	Need ML-based recommendations
Dynamic pricing	WooCommerce Dynamic Pricing (free)	Complex rule sets
Subscriptions	—	WooCommerce Subscriptions \$199/yr
Multi-vendor	—	Dokan / WC Vendors
AI catalog audit	WC AI Product Advisor (free official)	—

5. AI-Powered WooCommerce Workflows

5.1 AI Product Content Pipeline

Google Sheets / Airtable (product specs)

↓

WP AI Client / OpenAI API (generate SEO description)

↓

WP AI Client / FAL.ai (generate product images)

↓

WooCommerce REST API (create product programmatically)

↓

ACF Fields (auto-populate specs from AI analysis)

↓

Yoast SEO (auto-optimize with AI keywords)

5.2 AI Customer Service Stack

Customer message (web chat / social / email)

↓

Helpmate / AI chatbot (first response, product lookup)

↓

WooCommerce REST API (check order status, inventory)

↓

If unresolved → Human agent + AI-suggested response

↓

AI Puffer (automated follow-up emails based on interaction)

5.3 AI Inventory Management

// AI-powered low-stock prediction and reorder suggestions

function ai_inventory_analysis() {

\$products = wc_get_products([

'limit' => -1,

'status' => 'publish',

'stock_status' => 'instock',

]);

\$inventory_data = [];

foreach (\$products as \$product) {

\$orders_30d = wc_get_orders([

'limit' => -1,

'status' => ['completed'],

'date_created' => '>=' . date('Y-m-d', strtotime('-30 days')),

'meta_key' => '_product_id',

'meta_value' => \$product->get_id(),

]);

\$inventory_data[] = [

'name' => \$product->get_name(),

'stock' => \$product->get_stock_quantity(),

'sales_30d' => count(\$orders_30d),

'daily_avg' => round(count(\$orders_30d) / 30, 1),

'days_remaining'=> \$product->get_stock_quantity() / max(1, round(count(\$orders_30d) / 30, 1)),

];

}

\$analysis = wp_ai_client_prompt(

'Analyze this WooCommerce inventory data and flag products that need reordering within 14 days. Return JSON array with product_name, urgency (high/medium/low), and suggested_reorder_quantity'.

)->as_json_response([

'products' => ['type' => 'array', 'items' => ['type' => 'object']]

])->generate_text();

return \$analysis;

}

5.4 AI Fraud Detection Pattern

// AI-powered order fraud scoring

function ai_fraud_score(\$order_id) {

\$order = wc_get_order(\$order_id);

\$risk_factors = [

'billing_phone' => \$order->get_billing_phone(),

'billing_email' => \$order->get_billing_email(),

'shipping_country'=> \$order->get_shipping_country(),

'billing_country'=> \$order->get_billing_country(),

'total' => \$order->get_total(),

'items_count' => count(\$order->get_items()),

];

}


```
'payment_method' => $order->get_payment_method(),
'ip_address'      => $order->get_customer_ip_address(),
'is_first_order'  => wc_get_customer_order_count($order->get_customer_id()) <= 1,
'shipping_mismatch' => $order->get_billing_country() !== $order->get_shipping_country(),
];

$score = wp_ai_client_prompt(
    'Score this WooCommerce order for fraud risk on a scale of 1-100. Consider: phone format, email domain, country mismatch, high total, first order, IP. Return JSON: {"score": <int>}'
->using_temperature(0.3)->generate_text();

return json_decode($score, true);
}
```

6. Security Best Practices

6.1 WooCommerce Security Checklist

```
// 1. Force HTTPS for checkout
define('FORCE_SSL_ADMIN', true);
if (strpos($_SERVER['HTTP_X_FORWARDED_PROTO'] ?? '', 'https') !== false) {
    $_SERVER['HTTPS'] = 'on';
}

// 2. Disable REST API for unauthorized users (keep WooCommerce endpoints)
add_filter('woocommerce_rest_check_permissions', function($permission, $context, $object_id, $post_type) {
    if (!is_user_logged_in() && $context === 'read') {
        // Only allow read access to products/categories for public catalog
        if ($post_type === 'product') {
            return true;
        }
        return false;
    }
    return $permission;
}, 10, 4);

// 3. Remove WooCommerce version from frontend
remove_action('wp_head', 'wc_generator_tag');

// 4. Disable user enumeration
add_filter('rest_endpoints', function($endpoints) {
    if (isset($endpoints['/wp/v2/users'])) {
        unset($endpoints['/wp/v2/users']);
    }
    return $endpoints;
});

// 5. Harden payment processing
add_filter('woocommerce_payment_complete_order_status', function($status, $order_id) {
    // Always log payment completion
    $order = wc_get_order($order_id);
    error_log("Payment completed for order #{$order_id} - Total: {$order->get_total()}");
    return $status;
}, 10, 2);
```

6.2 SQL Injection Prevention

```
// ■ NEVER do this:
global $wpdb;
$order_id = $_GET['order_id'];
$result = $wpdb->get_results("SELECT * FROM {$wpdb->prefix}posts WHERE ID = $order_id");

// ■ Always use WooCommerce CRUD or $wpdb->prepare():
$order = wc_get_order(absint($_GET['order_id'])); // Auto-sanitized
$result = $wpdb->get_results($wpdb->prepare(
    "SELECT * FROM {$wpdb->prefix}wc_orders WHERE id = %d",
    absint($_GET['order_id'])
));
```

6.3 Data Sanitization & Validation

```
// Validate and sanitize all inputs
$product_id = isset($_POST['product_id']) ? absint($_POST['product_id']) : 0;
$quantity   = isset($_POST['quantity']) ? absint($_POST['quantity']) : 1;
$note       = isset($_POST['note']) ? sanitize_text_field($_POST['note']) : '';

// Nonce verification
if (!wp_verify_nonce($_POST['_wpnonce'], 'my-woo-action')) {
    wp_die('Security check failed');
}

// Capability checks
if (!current_user_can('edit_shop_orders')) {
    wp_die('Insufficient permissions');
}

// Output escaping
echo esc_html($order->get_billing_first_name());
echo wp_kses_post($product_description);
```

6.4 Recommended Security Plugins

Plugin	Free/Premium	Purpose
Wordfence	Free/Premium	WAF, malware scan, login security
Solid Security (iThemes)	Free/Premium	Brute force, 2FA, file change detection
WP Activity Log	Free/Premium	Audit logging (track who changed what)
Sucuri	Premium	Cloud WAF + CDN + monitoring
Really Simple SSL	Free/Premium	SSL enforcement + mixed content fixer

7. Performance Optimization

7.1 WooCommerce Performance Stack

```
CDN (Cloudflare / BunnyCDN)
↓
Page Cache (WP Super Cache / W3 Total Cache / WP Rocket)
↓
Object Cache (Redis via Redis Object Cache plugin)
↓
Database Optimization (HPOS enabled, clean postmeta)
↓
Image Optimization (WebP conversion, lazy loading)
```

↓
Code Optimization (minimal plugins, efficient queries)

7.2 Critical Performance Configurations

```
// Disable WooCommerce cart fragments on non-cart pages
add_action('wp_enqueue_scripts', function() {
    if (!is_cart() && !is_checkout()) {
        wp_dequeue_script('wc-cart-fragments');
        wp_dequeue_script('wc-add-to-cart');
    }
}, 99);

// Limit order history queries
add_filter('woocommerce_my_account_my_orders_query', function($args) {
    $args['limit'] = 10;
    $args['paginate'] = true;
    return $args;
});

// Optimize product queries
add_filter('woocommerce_product_query_meta_query', function($meta_query) {
    // Remove unnecessary meta queries
    return $meta_query;
});

// Disable WooCommerce admin for non-admin users
add_action('admin_init', function() {
    if (!current_user_can('manage_woocommerce') && !wp_doing_ajax()) {
        // Don't load admin assets for non-WooCommerce roles
    }
});
```

7.3 HPOS Performance Benefits

Enabling HPOS (default on new installs) provides: - **40-60% faster** order queries (no postmeta JOINS) - **Indexed queries** on order tables (wc_orders, wc_order_addresses, wc_order_operational_data) - **No wp_posts bloat** from shop_order post types - **Cleaner database** (orders don't pollute posts table)

7.4 Caching Considerations for WooCommerce

```
// Pages that should NEVER be cached:
// - /cart/
// - /checkout/
// - /my-account/*
// - /wc-api/* (payment gateway callbacks)

// In WP Rocket / W3TC, add these exclusions:
// Cart page: is_cart()
// Checkout page: is_checkout()
// Account pages: is_account_page()
// AJAX requests: defined('DOING_AJAX')

// WooCommerce cookie-based cache busting:
// Cookie: woocommerce_items_in_cart
// Cookie: woocommerce_cart_hash
// These cookies tell caching layers when to bypass cache
```

7.5 Image Optimization for Product Images

```
# Batch convert product images to WebP using WP-CLI
wp media regenerate --yes --only-missing

# Use WebP Express or Imagify for automatic conversion
# Recommended sizes for product images:
# - Catalog thumbnail: 300x300
# - Single product: 600x600
# - Full (zoom): 1200x1200 (max)

# Optimize with imagemin / squoosh
# Or use CloudFlare Polish for automatic image optimization at CDN level
```

8. Deployment & Hosting

8.1 WooCommerce Hosting Requirements

Requirement	Minimum	Recommended
PHP Version	7.4	8.1+
MySQL	5.7+	8.0+ (required for HPOS indexes)
Memory Limit	256MB	512MB+
Max Execution	60s	300s
Max Input Vars	1000	3000+ (large product variations)
WP Memory Limit	128MB	256MB+
Storage	SSD	NVMe SSD

8.2 Hostinger WooCommerce Compatibility

Hostinger is fully compatible with WooCommerce. Use the Business or Cloud plan for WooCommerce stores.

```
# Hostinger WP-CLI access (via SSH)
wp wc cot enable # Enable HPOS
wp wc cot migrate # Migrate existing orders

# Hostinger-specific optimizations:
# - Built-in LiteSpeed Cache (use LSCache plugin)
# - Redis available on Business/Cloud plans
# - Automatic daily backups included
# - Free SSL (Let's Encrypt)
# - CDN included on Cloud plans
```

8.3 Deployment Pipeline

```
LocalWP / EnvKit (development)
↓ Git push
GitHub (version control)
↓ GitHub Actions / WP-CLI
```

```
Staging (Hostinger subdomain)
↓ Testing + client approval
Production (Hostinger main domain)
```

8.4 Backup Strategy

Three-tier backup: 1. **Host-level:** Hostinger automatic daily backups (30-day retention on Business plan) 2. **Plugin-level:** UpdraftPlus → Google Drive / Dropbox (configurable schedule) 3. **Manual:** WP-CLI export before major changes

```
# Manual WooCommerce data backup
wp db export --tables=$(wp db tables --all-tables-with-prefix --format=csv | grep 'woocommerce\|wc_' | tr '\n' ',') backups/woocommerce-$(date +%Y%m%d).sql

# Full site export
wp export --dir=backups/ --filename_format=full-site-{date}.xml
```

8.5 Production Hardening Checklist

```
// wp-config.php production settings
define('WP_DEBUG', false);
define('WP_DEBUG_LOG', false);
define('WP_DEBUG_DISPLAY', false);
define('DISALLOW_FILE_EDIT', true);
define('DISALLOW_FILE_MODS', true); // Disable plugin/theme install from admin
define('AUTOMATIC_UPDATER_DISABLED', true); // Use managed updates
define('WP_POST_REVISIONS', 5); // Limit revisions
define('EMPTY_TRASH_DAYS', 7); // Auto-empty trash
define('WP_MEMORY_LIMIT', '256M');
define('WP_MAX_MEMORY_LIMIT', '512M');

// WooCommerce-specific
define('WC_LOG_DIR', WP_CONTENT_DIR . '/uploads/wc-logs/');
define('WC_LOG_HANDLER', 'WC_Log_Handler_File');
```

9. WP-CLI WooCommerce Commands

9.1 Essential WooCommerce WP-CLI

```
# Products
wp wc product list           # List all products
wp wc product create --name="Test Product" --regular_price="19.99" --sku="SKU001"
wp wc product update 123 --stock_quantity=50
wp wc product delete 123     # Move to trash
wp wc product generate       # Generate sample products (WooCommerce Smooth Generator)

# Orders
wp wc shop_order list --status=processing
wp wc shop_order get 456
wp wc shop_order update 456 --status=completed
wp wc shop_order generate --count=100    # Generate test orders

# Customers
wp wc customer list
wp wc customer get 789
wp wc customer create --email="test@example.com" --first_name="John" --last_name="Doe"

# Coupons
wp wc shop_coupon list
wp wc shop_coupon create --code="SAVE10" --discount_type="percent" --amount=10

# HPOS
wp wc cot enable             # Enable HPOS
wp wc cot enable --sync      # Enable with sync mode
wp wc cot migrate            # Migrate existing orders
wp wc cot count              # Count orders in both tables
wp wc cot verify             # Verify migration

# Tools
wp wc tool run install_pages # Create default WC pages
wp wc tool run clear_transients # Clear WC transients
wp wc tool run clear_expired_transients
wp wc tool run recount_terms # Recount product category terms
wp wc tool run update_product_lookup_tables # Regenerate product lookup tables
wp wc tool run clear_customer_sessions # Clear all customer sessions

# System
wp wc status                 # WooCommerce system status
wp wc system_status          # Detailed system report
wp wc api_keys list          # List REST API keys
wp wc shipping_zone list     # List shipping zones
wp wc tax list               # List tax rates

# Settings
wp wc setting get woocommerce_store_address
wp wc setting update woocommerce_currency USD
wp wc setting update woocommerce_weight_unit kg
wp wc setting update woocommerce_dimension_unit cm
```

9.2 ACF + WooCommerce WP-CLI

```
# Sync ACF JSON field groups (includes product field groups)
wp acf sync

# Export ACF field groups to JSON
wp acf export

# Regenerate ACF local JSON
wp acf clean
```

10. Setup Checklist

New WooCommerce Store Setup

- [] Install WordPress 7.0.2 on PHP 8.1+
- [] Install and activate WooCommerce latest (11.1+)
- [] Run WooCommerce setup wizard (store location, currency, payment methods)
- [] Enable HPOS: `wp wc cot enable`
- [] Create default pages: `wp wc tool run install_pages`
- [] Configure permalinks (Post name)
- [] Set up payment gateway(s) — Stripe + PayPal

- ☐ Configure shipping zones and rates
- ☐ Set up tax rules (or install TaxJar / Avalara)
- ☐ Install and configure caching plugin (WP Super Cache / LSCache)
- ☐ Install Wordfence security
- ☐ Configure backups (UpdraftPlus + offsite storage)
- ☐ Set up Google Analytics / WooCommerce Analytics
- ☐ Create essential pages (Shop, Cart, Checkout, My Account, Terms, Privacy)
- ☐ Configure transactional email settings + SMTP
- ☐ Set up image optimization (WebP Express + lazy loading)
- ☐ Test entire purchase flow (add to cart → checkout → payment → order confirmation)
- ☐ Verify mobile responsiveness
- ☐ Configure CDN (Cloudflare free tier)
- ☐ Set `WP_DEBUG=false` and `DISALLOW_FILE_EDIT=true` in wp-config.php

AI Integration Setup

- ☐ Configure WP 7.0 Connectors (Settings > Connectors — add Anthropic/OpenAI keys)
- ☐ Install WooCommerce AI Product Advisor plugin
- ☐ Set up n8n self-hosted with WooCommerce node
- ☐ Create REST API keys with read/write permissions for AI agents
- ☐ Configure WP AI Client for product description generation
- ☐ Set up AI Puffer or AI Bud for content automation
- ☐ Create AI-generated product template with ACF fields
- ☐ Configure product image alt text auto-generation

Divi 5 + WooCommerce Setup

- ☐ Install Divi 5.1 theme (latest)
- ☐ Create Divi child theme
- ☐ Install Squad Modules Lite (65 free Divi 5 modules) or Divi Torque Lite
- ☐ Run divi5-skill initialization: `git clone https://github.com/divilovewp/divi5-skill.git` and attach SKILL.md to AI tooling
- ☐ Set up Theme Builder templates:
- ☐ Product single template (with WooCommerce element modules)
- ☐ Shop/Archive template (with `divi/shop` grid)
- ☐ Cart template (with cart modules)
- ☐ Checkout template (with checkout modules)
- ☐ My Account template
- ☐ Design header/footer in Theme Builder (global)
- ☐ Verify all 25 WooCommerce modules render correctly with test products
- ☐ Test Loop Builder with WooCommerce products
- ☐ Set up IA Webmaster Bridge or DiviOps for AI-assisted building
- ☐ Configure cart/checkout page assignments in WooCommerce settings

Reference Links — Complete Index

Official WooCommerce

Resource	URL
Developer Docs	https://developer.woocommerce.com/docs/
REST API Docs	https://woocommerce.com/document/woocommerce-rest-api/
Code Reference	https://woocommerce.github.io/code-reference/
GitHub Monorepo	https://github.com/woocommerce/woocommerce
Storybook	https://woocommerce.github.io/woocommerce/
AI Developer Guide	https://developer.woocommerce.com/docs/getting-started/ai/
HPOS Docs	https://developer.woocommerce.com/docs/features/orders/high-performance-order-storage/
MCP Integration Docs	https://developer.woocommerce.com/docs/features/mcp-integration/
Slack Community	https://woocommerce.com/community-slack/

MCP Servers & AI Tools

Resource	URL
woocommerce-mcp-server	https://github.com/xmichal88x/woocommerce-mcp-server
agentpress (WP + DB MCP)	https://github.com/multideskio/agentpress
Agent2Wp	https://github.com/Marksab001/Agent2Wp
IA Webmaster Bridge (Divi 5 MCP)	https://github.com/RiusmaX/ia-webmaster-bridge
divi5-skill (JSON-native)	https://github.com/divilovewp/divi5-skill
DiviOps (Divi 5 MCP)	https://github.com/oaris-dev/diviops
acf-mcp-server	https://github.com/symonbaikov/acf-mcp-server
UCP Checkout for Woo	https://github.com/boxybird/ucp-checkout-for-woocommerce

AI Plugins

Plugin	URL
WC AI Product Advisor	https://github.com/woocommerce/woocommerce-ai-product-advisor
CopyCraft	https://github.com/tectalic/CopyCraft
Helpmate	https://github.com/rhapsody-plugins/helpmate
AI Puffer	https://wordpress.org/plugins/ai-puffer/
AI Bud	https://wordpress.org/plugins/ai-bud/

Dominopost	https://wordpress.org/plugins/dominopost/
------------	---

Divi 5

Resource	URL
Divi 5 Docs (Official)	https://help.elegantthemes.com/en/collections/10650977-divi-5
Divi 5 Community Docs	https://16wells.github.io/divi-docs/
Loop Builder Guide	https://diviflash.com/divi-5-loop-builder/
Squad Modules Lite	https://github.com/thewpsquad/squad-modules-lite
Divi Torque Lite	https://wordpress.org/plugins/divi-torque-lite/

Development Tools

Resource	URL
LocalWP	https://localwp.com/
EnvKit (LocalWP alt)	https://github.com/Env-Kit/envkit-releases
Novamira (WP MCP)	https://github.com/use-novamira/novamira
wpmcp (safe WP MCP)	https://github.com/wpmcp/wpmcp
WP-CLI	https://wp-cli.org/
HPOS Compatibility Scanner	https://github.com/dustinparker/hpos-compatibility-scanner

Hosting & Performance

Resource	URL
Hostinger	https://www.hostinger.com/ (WooCommerce-compatible, Redis/LSCache)
Cloudflare	https://www.cloudflare.com/ (free CDN + WAF)
WP Rocket	https://wp-rocket.me/ (premium caching)
Redis Object Cache	https://wordpress.org/plugins/redis-cache/

Security

Resource	URL
Wordfence	https://wordpress.org/plugins/wordfence/
Solid Security	https://wordpress.org/plugins/better-wp-security/
Sucuri	https://sucuri.net/

■ PART B: E-COMMERCE ECOSYSTEM — Payments, Shipping, Tax & Automation

WooCommerce E-Commerce Ecosystem Playbook — Payments, Shipping, Tax & AI Automation

Compiled: July 17, 2026 | **Companion to:** [woocommerce-ai-dev-playbook.md](#) **Scope:** Complete payment gateway ecosystem, shipping carrier integrations, tax automation, bulk AI product creation from images, agentic commerce protocols, n8n workflows, and store bootstrap automation.

Table of Contents

- 1. [Payment Gateways Ecosystem](#)
- 2. [Shipping & Fulfillment](#)
- 3. [Tax Automation](#)
- 4. [AI Bulk Product Creation from Images](#)
- 5. [AI-Powered Store Automation Workflows](#)
- 6. [Agentic Commerce Protocols](#)
- 7. [MCP Servers for E-Commerce](#)
- 8. [n8n WooCommerce Automation Templates](#)
- 9. [Store Setup Automation & Bootstrap](#)
- 10. [Reference Index](#)

1. Payment Gateways Ecosystem

1.1 Stripe — The Gold Standard

Official WooCommerce Stripe Gateway - **Repo:** [woocommerce/woocommerce-gateway-stripe](#) — ■288, Official Automattic plugin - **WP.org:** [woocommerce-gateway-stripe](#) - **Payment methods:** Credit/debit cards, Apple Pay, Google Pay, Link (Stripe's accelerated checkout), SEPA, iDEAL, Bancontact, Sofort, giropay, P24, EPS, Alipay, WeChat Pay, Klarna, Afterpay/Clearpay, Affirm - **Features:** SCA/3D Secure 2, webhook handling, payment intents, refunds from WP admin, subscription tokenization, express checkout (WooPay/Stripe Link) - **Webhook events:** `payment_intent.succeeded`, `payment_intent.payment_failed`, `charge.refunded`, `checkout.session.completed`, `customer.subscription.*` - **PHP SDK:** `stripe/stripe-php` (Composer) — `composer require stripe/stripe-php` - **WooCommerce REST API integration:** Stripe handles payment on checkout POST; gateway is selectable via `payment_method: "stripe"` in REST API - **PCI Compliance:** Stripe Elements/Checkout embeds an iframe — card data never touches the WP server. SAQ A-EP.

Stripe AI Toolkit & MCP - **Official Stripe Agent Toolkit:** [stripe/ai](#) — Stripe's official AI tooling for building payment agents - **Official Stripe MCP Server:** [docs.stripe.com/mcp](#) — Remote MCP server exposing Stripe payment operations to AI assistants - **Stripe MPP (Machine Payments Protocol):** Stripe + Tempo Labs co-authored HTTP 402-based machine-to-machine payments standard. See [stripe-mpp plugin](#)

Stripe WooCommerce Setup (WP-CLI)

```
# Install from WordPress.org
wp plugin install woocommerce-gateway-stripe --activate
```

```
# Configure via WP-CLI
wp option update woocommerce_stripe_settings '{
  "enabled": "yes",
  "title": "Credit Card (Stripe)",
  "description": "Pay with credit card via Stripe",
  "testmode": "no",
  "publishable_key": "pk_live_xxx",
  "secret_key": "sk_live_xxx",
  "webhook_secret": "whsec_xxx",
  "statement_descriptor": "MYSTORE",
  "capture": "yes",
  "payment_request": "yes",
  "saved_cards": "yes"
}' --format=json
```

Stripe Webhook Registration

```
// Handle Stripe webhook events
add_action('woocommerce_api_wc_gateway_stripe', function() {
    $raw_body = file_get_contents('php://input');
    $sig_header = $_SERVER['HTTP_STRIPE_SIGNATURE'] ?? '';
    $secret = 'whsec_YOUR_WEBHOOK_SECRET';

    // Verify signature
    try {
        $event = \Stripe\Webhook::constructEvent($raw_body, $sig_header, $secret);
    } catch(\Exception $e) {
        http_response_code(400);
        exit();
    }

    switch ($event->type) {
        case 'payment_intent.succeeded':
            $intent = $event->data->object;
            // Handle successful payment
            break;
        case 'charge.refunded':
            $charge = $event->data->object;
            // Handle refund
            break;
    }

    http_response_code(200);
});
```

1.2 PayPal — Full Suite

Official WooCommerce PayPal Payments - WP.org: [woocommerce-paypal-payments](#) - **Features:** PayPal Checkout, Smart Buttons, PayPal Credit, Pay Later, Venmo (US), credit/debit cards, advanced card processing, vaulting (saved payments), subscription support - **REST API:** [developer.paypal.com](#) — Orders API v2, Subscriptions API, Disputes API, Payouts API, Invoicing API

Official PayPal MCP Server - Repo: [paypal/paypal-mcp-server](#) — **■11**, JavaScript - **Package:** `npx -y @paypal/mcp --tools=all` - **Auth:** `PAYPAL_ACCESS_TOKEN + PAYPAL_ENVIRONMENT=SANDBOX|PRODUCTION`

Available MCP Tools (Official PayPal) | Tool Category | Tools | `---` | **Invoices** | `create_invoice, list_invoices, get_invoice, send_invoice, send_invoice_reminder, cancel_sent_invoice, generate_invoice_qr_code` | **Payments** | `create_order, get_order, pay_order, create_refund, get_refund` | **Disputes** | `list_disputes, get_dispute, accept_dispute_claim` | **Shipment Tracking** | `create_shipment_tracking, get_shipment_tracking` | **Catalog** | `create_product, list_products, show_product_details, update_product` | **Subscriptions** | `create_subscription_plan, update_plan, list_subscription_plans, show_subscription_plan_details, create_subscription, show_subscription_details, update_subscription, cancel_subscription` | **Reporting** | `list_transactions`

MCP Configuration for Hermes (Claude Desktop / Cursor)

```
{
  "mcpServers": {
    "paypal": {
      "command": "npx",
      "args": ["-y", "@paypal/mcp", "--tools=all"],
      "env": {
        "PAYPAL_ACCESS_TOKEN": "<YOUR_ACCESS_TOKEN>",
        "PAYPAL_ENVIRONMENT": "SANDBOX"
      }
    }
  }
}
```

PayPal Access Token Generation

```
# OAuth 2.0 client credentials
curl -X POST https://api-m.sandbox.paypal.com/v1/oauth2/token \
-H "Accept: application/json" \
-H "Accept-Language: en_US" \
-u "CLIENT_ID:CLIENT_SECRET" \
-d "grant_type=client_credentials"
```

PayPal Agent Toolkit: [paypal/agent-toolkit](#) — Official PayPal toolkit for integrating commerce actions into AI agents.

1.3 Venmo

- **Integrated via:** WooCommerce PayPal Payments plugin — Venmo is a native payment method within PayPal Checkout
- **Availability:** US only, mobile-optimized
- **Setup:** Enable in PayPal Payments settings → Payment Methods → Venmo
- **No standalone WooCommerce plugin exists** — Venmo for business is exclusively through PayPal Checkout

1.4 Zelle

- **No native WooCommerce plugin exists on wp.org**
- **Options:**
- **Zelle via Stripe:** No — Stripe does not support Zelle
- **Zelle via Authorize.net:** No
- **Manual workaround:** Add a custom gateway with Zelle payment instructions (email/phone number display on thank you page), then manually verify
- **B2B alternative:** GoDaddy Payments includes Zelle-like bank transfers for WooCommerce (US only)

1.5 Cash App

- **WooCommerce Cash App Pay:** Available through Square + WooCommerce Square plugin

- Requires WooCommerce Square plugin + Square account
- Cash App Pay appears as payment method when Square is gateway
- **No standalone Cash App WooCommerce plugin exists on wp.org**
- **Alternative:** Stripe supports Cash App Pay in beta (US only) — enable via Stripe Dashboard → Payment Methods

1.6 WooPayments (Native Solution)

- **WP.org:** [woopayments](#) — formerly WooCommerce Payments
- **What it is:** WooCommerce's native payment processing built on Stripe infrastructure
- **Features:** In-dashboard management (no Stripe account needed), multi-currency, WooPay express checkout, fraud protection, deposits directly to bank, disputes management, subscriptions
- **Pricing:** 2.9% + \$0.30 per transaction (US) — competitive with Stripe direct
- **Key difference from standalone Stripe:** Management entirely within WP admin; no need to open Stripe dashboard; unified reporting

1.7 Payment Security: PCI Compliance

Compliance Level	Requirement	How to Achieve
SAQ A	Card data never touches your server	Stripe Checkout / Elements (iframe) or PayPal redirect
SAQ A-EP	Card data handled via direct post	Stripe Elements + proper CSP headers
SAQ D	Full PCI DSS	Self-hosted payment form (avoid this!)

WooCommerce PCI Best Practices:

```
// Force SSL on all pages
define('FORCE_SSL_ADMIN', true);

// Stripe handles card data in iframe - SAQ A-EP
// PayPal redirects - SAQ A
// Never store raw card numbers in wp_postmeta or custom tables

// Tokenize with payment gateway - never store CC numbers:
// ■ Stripe payment method token: pm_xxx
// ■ PayPal billing agreement: B-xxx
// ■ Raw CC: "4111 1111 1111 1111"
```

1.8 Multi-Currency Support

WooCommerce Multilingual (WPML): - **WP.org:** [woocommerce-multilingual](#) - **Features:** Multi-currency with fixed rates or auto-update from currency feeds, currency switcher widget, geo-location currency detection, per-product currency pricing - **WP-CLI for currency:** `wp wc setting update woocommerce_currency EUR`

WooCommerce Currency Switcher (Aelia): - Premium plugin for multi-currency without full multilingual setup - PayPal + Stripe multi-currency auto-detection

WooCommerce Payments Multi-Currency: - Built into WooPayments — accepts 135+ currencies, settles in your base currency

1.9 Subscriptions & Recurring Payments

WooCommerce Subscriptions (Official): - **Price:** \$199/yr from woocommerce.com - **Features:** Flexible billing schedules, automatic rebilling via 25+ payment gateways, free trials, sign-up fees, subscription management portal, proration, upgrades/downgrades, renewal notifications, churn-reduction emails

Free Alternatives: - **Subscriptions for WooCommerce** (WP Swings) — wp.org, freemium - **YITH WooCommerce Subscription** — wp.org, freemium

Subscription Payment Gateway Support: | Gateway | Recurring Support | Tokenization | |---|---| | Stripe (WC Stripe Gateway) | ■ Native | ■ Yes | | PayPal (WC PayPal Payments) | ■ Via Vaulting | ■ Yes | | WooPayments | ■ Native | ■ Yes | | Square | ■ Limited | ■ Yes | | Authorize.net | ■ CIM | ■ Yes |

WP-CLI Subscription Commands:

```
# List subscriptions
wp wc shop_subscription list --status=active

# Get subscription details
wp wc shop_subscription get 123

# Cancel a subscription
wp wc shop_subscription update 123 --status=cancelled
```

1.10 Payment Gateway Comparison Matrix

Gateway	Free Plugin	MCP Server	AI Toolkit	Venmo	Multi-Currency	Subscriptions
Stripe	■ (official)	■ Official	■ stripe/ai	■	■	■
PayPal	■ (official)	■ Official	■ paypal/agent-toolkit	■	■	■
WooPayments	■ (free plugin)	■	■	■	■	■
Square	■ (official)	■	■	■ (Cash App)	Limited	Limited
Braintree	■ (official)	■	■	■ (via PayPal)	■	■
Authorize.net	■ (official)	■	■	■	■	■
Mollie	■ (wp.org)	■	■	■	■	■
Klarna	■ (official)	■	■	■	■	■
Amazon Pay	■ (official)	■	■	■	■	■

2. Shipping & Fulfillment

2.1 WooCommerce Built-in Shipping

Shipping Zones: - Defined via WooCommerce > Settings > Shipping > Shipping Zones - Each zone: region(s) + shipping methods - Supports: countries, states/provinces, zip code ranges - Fallback: "Locations not covered by your other zones" zone

Shipping Classes: - Product-level classification (e.g., "Heavy", "Fragile", "Free Shipping") - Used in flat rate / table rate calculations

Built-in Shipping Methods: | Method | Description | Use Case | |---|---| | Flat Rate | Fixed cost per order/item/class | Predictable shipping | | Free Shipping | Free with coupon, minimum order, or always | Threshold promotions | | Local Pickup | Customer picks up at location | Physical store/location |

WP-CLI Shipping:

```
# List shipping zones
wp wc shipping_zone list

# Create a shipping zone
wp wc shipping_zone create --name="US Mainland" --order=1

# Add shipping method to zone
wp wc shipping_zone_method create <zone_id> --method_id="flat_rate" \
--method_title="Standard Shipping" \
--settings='{ "cost": "5.99", "tax_status": "taxable" }'
```

2.2 Live Carrier Rate Plugins

WooCommerce Shipping & Tax (Official — Automatic) - Repo: [Automatic/woocommerce-services](#) — ■112 - WP.org: [woocommerce-services](#) - Features: Automated tax rates, USPS live shipping rates, purchase & print USPS labels, package dimensions - Limitation: USPS only; for UPS/FedEx/DHL, use third-party

Third-Party Carrier Rate Plugins (wp.org)

Plugin	Carriers	Features	Free/Premium
WooCommerce Shipping (Official)	USPS	Labels, rates, tax	Free
Table Rate Shipping (Official WC)	Custom rules	Weight/count/class-based tables	\$99/yr
UPS Shipping Method (PluginHive)	UPS	Live rates, label printing, tracking	Freemium
FedEx Shipping Method (PluginHive)	FedEx	Live rates, label printing, tracking	Freemium
DHL Express (PluginHive)	DHL	Live rates, label printing, customs	Freemium
ELEX USPS Shipping	USPS	All USPS services, flat rate boxes	Freemium
Shippo	55+ carriers	Multi-carrier rates, labels, tracking	Pay-as-you-go
ShipStation	50+ carriers	Order management, batch labels, automation	\$9.99+/mo
Easyship	250+ carriers	International focus, duties/taxes calculator	Pay-as-you-go
Sendcloud	80+ carriers (EU)	Checkout rates, labels, returns, tracking	Freemium (EU-focused)

2.3 Shipping Label Printing Automation

WooCommerce Shipping (Official):

```
# Purchase USPS labels directly from WooCommerce admin
# Requires WooCommerce Shipping & Tax plugin + USPS account
# Dashboard: WooCommerce > Orders > Click order > "Create shipping label"
```

ShipStation Integration: - Webhook-based: ShipStation polls WooCommerce REST API for new orders - Auto-creates shipping labels, sends tracking numbers back to WooCommerce - Batch processing — print 100s of labels at once - Custom automation rules ("if order > \$50 → Signature Confirmation")

Shippo API for Custom Automation:

```
// Create shipping label via Shippo API from WordPress
function create_shipping_label($order_id) {
    $order = wc_get_order($order_id);
    $address = $order->get_address('shipping');

    $response = wp_remote_post('https://api.goshippo.com/shipments/', [
        'headers' => [
            'Authorization' => 'ShippoToken YOUR_API_KEY',
            'Content-Type' => 'application/json',
        ],
        'body' => json_encode([
            'address_from' => [
                'name' => 'My Store',
                'street1' => '123 Main St',
                'city' => 'New York',
                'state' => 'NY',
                'zip' => '10001',
                'country' => 'US',
            ],
            'address_to' => [
                'name' => $address['first_name'] . ' ' . $address['last_name'],
                'street1' => $address['address_1'],
                'city' => $address['city'],
                'state' => $address['state'],
                'zip' => $address['postcode'],
                'country' => $address['country'],
            ],
            'parcels' => [[
                'length' => '10',
                'width' => '8',
                'height' => '4',
                'distance_unit' => 'in',
                'weight' => '2',
                'mass_unit' => 'lb',
            ]],
            'async' => false,
        ]),
    ]);

    $shipment = json_decode(wp_remote_retrieve_body($response), true);

    if (!empty($shipment['rates'][0])) {
        $rate = $shipment['rates'][0]['object_id'];
        // Purchase label
        $label = wp_remote_post("https://api.goshippo.com/transactions/", [
            'headers' => ['Authorization' => 'ShippoToken YOUR_API_KEY'],
            'body' => json_encode(['rate' => $rate, 'async' => false]),
        ]);

        $transaction = json_decode(wp_remote_retrieve_body($label), true);
        $tracking_number = $transaction['tracking_number'] ?? '';
        $label_url = $transaction['label_url'] ?? '';

        // Save tracking to order
        $order->add_order_note("Tracking: $tracking_number");
        update_post_meta($order_id, '_tracking_number', $tracking_number);
        update_post_meta($order_id, '_shipping_label_url', $label_url);
    }
}
```

2.4 Shipping Carrier APIs (Direct)

Carrier	API	Developer Portal
USPS	USPS Web Tools API (REST)	https://developer.usps.com
UPS	UPS Shipping API (OAuth 2.0)	https://developer.ups.com

FedEx	FedEx REST API	https://developer.fedex.com
DHL	DHL Express API	https://developer.dhl.com
EasyPost	Multi-carrier REST API	https://www.easypost.com/docs
ShipEngine	Multi-carrier REST API	https://www.shipengine.com/docs
Shippo	Multi-carrier REST API	https://goshippo.com/docs

2.5 Shipping MCP Servers

shopanaio/carrier-api (■6) - Enterprise-grade shipping carrier API clients & MCP servers - [GitHub](#) — Nova Poshta / Ukraine carriers currently; expanding

DSers Official MCP Server (Dropshipping) - Remote MCP server for AI-powered dropshipping automation - Endpoint: <https://mcp.dsers.com/dropshipping/mcp> - OAuth 2.1 + PKCE auth (no API keys) - Features: Product import from AliExpress/Alibaba/1688/Accio, pricing rules, supplier replacement, store publishing - [GitHub Docs](#)

bizprint-mcp-server (■1) - Connect AI agents to physical printers — print receipts, shipping labels, packing slips - [GitHub](#)

2.6 Free Shipping Threshold Logic

```
// Dynamic free shipping threshold – AI-calculated
function dynamic_free_shipping_threshold() {
  // Get AOV for last 30 days
  $orders = wc_get_orders([
    'limit' => -1,
    'status' => ['completed'],
    'date_created' => '>=' . date('Y-m-d', strtotime('-30 days')),
  ]);

  $totals = array_map(function($order) { return $order->get_total(); }, $orders);
  $aov = count($totals) > 0 ? array_sum($totals) / count($totals) : 0;

  // Set threshold at AOV * 1.3 (encourage 30% higher cart)
  $threshold = round($aov * 1.3, -1); // Round to nearest 10

  return max($threshold, 50); // Minimum $50
}

add_filter('woocommerce_shipping_free_shipping_is_available', function($is_available, $package) {
  $threshold = dynamic_free_shipping_threshold();
  if (WC()->cart->get_subtotal() < $threshold) {
    return false;
  }
  return $is_available;
}, 10, 2);
```

2.7 Dropshipping Integration Patterns

DSers MCP Server for AI-Driven Dropshipping:

```
// Claude Desktop config
{
  "mcpServers": {
    "dsers": {
      "type": "http",
      "url": "https://mcp.dsers.com/dropshipping/mcp"
    }
  }
}
```

Fully Automated Dropshipping Pipeline (n8n): 1. Google Trends → trending keywords 2. AliExpress API → matching products 3. OpenAI/GPT → SEO-optimized titles + descriptions 4. WooCommerce REST API → auto-publish product 5. DSers → order fulfillment automation

Repos: - [a-nayem/Dropshipping-automation](#) — n8n pipeline - [shauncuier/dropshipzone](#) — WooCommerce plugin for Dropshipzone API - [ramphor/dropshipping](#) — WP/WooCommerce based dropshipping platform

2.8 Inventory Sync Across Locations

Multi-Location Inventory Plugins: | Plugin | Price | Features | [---][---][---] | ATUM Inventory Management | Free wp.org | Multi-warehouse, purchase orders, stock central | | Square for WooCommerce | Free | POS sync, location-based inventory | | TradeGecko (QuickBooks) | Premium | Multi-warehouse, forecasting, purchase orders | | Zoho Inventory | Freemium | Multi-warehouse, serial tracking, dropshipping |

Inventory REST API Pattern:

```
# Update stock quantity at warehouse level (via custom meta)
curl -X PUT "https://store.com/wp-json/wc/v3/products/123" \
-u "ck_xxx:cs_xxx" \
-H "Content-Type: application/json" \
-d '{
  "stock_quantity": 45,
  "meta_data": [
    {"key": "warehouse_a_stock", "value": "20"},
    {"key": "warehouse_b_stock", "value": "15"},
    {"key": "warehouse_c_stock", "value": "10"}
  ]
},'
```

3. Tax Automation

3.1 WooCommerce Built-in Tax

Core Tax Settings: - WooCommerce > Settings > Tax - Enable taxes - Prices entered with tax (inclusive) or without (exclusive) - Display prices in shop incl/excl tax - Display prices in cart/checkout incl/excl tax

Built-in Tax Rate Tables: - WooCommerce > Settings > Tax > Standard Rates (and Reduced Rate, Zero Rate) - Manual CSV import: Country, State, ZIP, City, Rate, Tax Name, Priority, Compound, Shipping - No automatic rate updates — static until manually changed

Tax Classes: - Standard (default), Reduced Rate, Zero Rate - Custom classes: create unlimited (e.g., "Digital Goods", "Food", "Clothing") - Assign to products individually

3.2 WooCommerce Tax (Automated — Official)

Part of WooCommerce Shipping & Tax: - WP.org: [woocommerce-services](#) - What it does: Auto-calculates sales tax rates at checkout based on customer address - Coverage: US sales tax, Canadian GST/HST/PST, Australian GST, EU VAT (selected countries) - Setup: WooCommerce > Settings > Tax > Automated Taxes (check box) - Cost: Free (rates provided as service by Automatic)

3.3 TaxJar Integration

TaxJar for WooCommerce: - **Repo:** [taxjar/taxjar-woocommerce-plugin](#) — **WP.org:** taxjar-woocommerce-plugin - **Features:** Real-time sales tax calculation at checkout, automated filing in 30+ states, economic nexus tracking, product taxability rules, exemption certificate management - **Pricing:** Starts at \$19/month for calculations; AutoFile adds cost - **API:** REST — <https://api.taxjar.com/v2/>

TaxJar API Quickstart:

```
# Calculate tax for an order
curl -X POST "https://api.taxjar.com/v2/taxes" \
-H "Authorization: Bearer YOUR_API_KEY" \
-H "Content-Type: application/json" \
-d '{
  "from_country": "US",
  "from_zip": "92093",
  "from_state": "CA",
  "to_country": "US",
  "to_zip": "90002",
  "to_state": "CA",
  "amount": 16.50,
  "shipping": 1.5,
  "line_items": [
    { "id": "1", "quantity": 1, "product_tax_code": "20010", "unit_price": 15.0, "discount": 0 }
  ]
}'
```

3.4 Avalara Integration

AvaTax for WooCommerce: - Official plugin on wp.org (Avalara) - **Features:** 12,000+ tax jurisdictions, real-time calculation, automated filing, exemption certificates, international VAT/GST - **Pricing:** Premium — contact sales; API-based - **API:** REST — <https://developer.avalara.com>

3.5 VAT, GST & International Tax

WooCommerce EU VAT Compliance: - **EU VAT Number Validation:** Built-in to WC core — validates VIES VAT numbers on checkout - **VAT MOSS/OSS:** Manual setup with tax rates; or use TaxJar/Avalara for automation - **UK VAT:** Post-Brexit — treat as import/export; WooCommerce + Avalara recommended

International Tax Plugins: | Plugin | Coverage | Features | |---|---|---| | WooCommerce Shipping & Tax | US, CA, AU, EU | Auto-calc, free | | TaxJar | US + International | Auto-file, nexus tracking | | Avalara AvaTax | Global (12K+ jurisdictions) | Enterprise-grade | | Quaderno | Global | Digital goods focus, automatic receipts | | TaxCloud | US only | Free for basic, paid for AutoFile |

3.6 B2B Tax Exemption

WooCommerce B2B Exemption Pattern:

```
// Remove tax for B2B customers with valid VAT number
add_filter('woocommerce_product_get_tax_class', function($tax_class, $product) {
    if (is_user_logged_in() && get_user_meta(get_current_user_id(), 'b2b_tax_exempt', true)) {
        return 'Zero Rate';
    }
    return $tax_class;
}, 10, 2);

// Validate EU VAT number on registration
add_action('woocommerce_register_form', function() {
    woocommerce_form_field('b2b_vat_number', [
        'type' => 'text',
        'label' => 'VAT Number (for B2B tax exemption)',
        'required' => false,
    ]);
});

add_action('woocommerce_created_customer', function($customer_id) {
    if (!empty($_POST['b2b_vat_number'])) {
        $vat = sanitize_text_field($_POST['b2b_vat_number']);
        // Validate against VIES database
        $valid = validate_eu_vat($vat);
        if ($valid) {
            update_user_meta($customer_id, 'b2b_tax_exempt', true);
            update_user_meta($customer_id, 'b2b_vat_number', $vat);
        }
    }
});
```

3.7 Tax Reporting Automation

```
// Generate monthly tax report via REST API
function generate_tax_report($month, $year) {
    $orders = wc_get_orders([
        'limit' => -1,
        'status' => ['completed', 'processing'],
        'date_created' => "{$year}-{$month}-01...{$year}-{$month}-31",
    ]);

    $report = [
        'period' => "{$year}-{$month}",
        'total_orders' => count($orders),
        'total_revenue' => 0,
        'total_tax' => 0,
        'total_shipping' => 0,
        'by_state' => [],
    ];

    foreach ($orders as $order) {
        $state = $order->get_shipping_state() ? $order->get_billing_state();
        $report['total_revenue'] += $order->get_total();
        $report['total_tax'] += $order->get_total_tax();
        $report['total_shipping'] += $order->get_shipping_total();

        if (!isset($report['by_state'][$state])) {
            $report['by_state'][$state] = ['orders' => 0, 'revenue' => 0, 'tax' => 0];
        }
        $report['by_state'][$state]['orders']++;
        $report['by_state'][$state]['revenue'] += $order->get_total();
        $report['by_state'][$state]['tax'] += $order->get_total_tax();
    }

    return $report;
}
```

4. AI Bulk Product Creation from Images

4.1 The Core Pattern: Image Folder → AI → WooCommerce Products



4.2 Implementation: Complete PHP Script

```

<?php
/**
 * AI Bulk Product Creator – Image Folder → WooCommerce
 * Dependencies: WP 7.0+, WooCommerce 11+, OpenAI/Claude API
 *
 * Usage: wp eval-file ai-bulk-product-creator.php
 */

class AI_Bulk_Product_Creator {
    private $image_dir;
    private $woocommerce;
    private $ai_endpoint = 'https://api.openai.com/v1/chat/completions'; // or Claude

    public function __construct() {
        $this->image_dir = WP_CONTENT_DIR . '/uploads/bulk-import/';
        $this->init_woocommerce_api();
    }

    /**
     * Scan directory for images and process each
     */
    public function scan_and_create() {
        if (!is_dir($this->image_dir)) {
            mkdir($this->image_dir, 0755, true);
            WP_CLI::log("Created directory: {$this->image_dir}");
            return;
        }

        $images = glob($this->image_dir . '.*.{jpg,jpeg,png,webp,gif}', GLOB_BRACE);
        $count = 0;

        foreach ($images as $image_path) {
            WP_CLI::log("Processing: " . basename($image_path));

            // Step 1: Upload image to WordPress media library
            $attachment_id = $this->upload_image($image_path);
            if (!$attachment_id) continue;

            // Step 2: AI vision analysis of the image
            $analysis = $this->analyze_image($attachment_id);
            if (!$analysis) continue;

            // Step 3: Generate product data with AI
            $product_data = $this->generate_product_data($analysis);
            if (!$product_data) continue;

            // Step 4: Create WooCommerce product
            $product_id = $this->create_product($product_data, $attachment_id);

            if ($product_id) {
                WP_CLI::success("Created product #{$product_id}: {$product_data['title']}");
                $count++;
            }
        }

        WP_CLI::success("Created {$count} products from images.");
    }

    /**
     * Upload image from directory to WordPress media library
     */
    private function upload_image($file_path) {
        $filename = basename($file_path);
        $upload_file = wp_upload_bits($filename, null, file_get_contents($file_path));

        if ($upload_file['error']) {
            WP_CLI::warning("Upload failed: {$filename}");
            return false;
        }

        $attachment = [
            'post_mime_type' => $upload_file['type'],
            'post_title' => sanitize_file_name(pathinfo($filename, PATHINFO_FILENAME)),
            'post_content' => '',
            'post_status' => 'inherit',
        ];

        $attachment_id = wp_insert_attachment($attachment, $upload_file['file']);
        require_once(ABSPATH . 'wp-admin/includes/image.php');
        $attach_data = wp_generate_attachment_metadata($attachment_id, $upload_file['file']);
        wp_update_attachment_metadata($attachment_id, $attach_data);

        return $attachment_id;
    }

    /**
     * AI vision analysis – describe the product from image
     */
    private function analyze_image($attachment_id) {
        $image_url = wp_get_attachment_url($attachment_id);

        // Using WordPress 7.0 AI Client (vision-enabled)
    }
}

```

```

if (function_exists('wp_ai_client_prompt')) {
    $prompt = "Analyze this product image. Describe: what is the product?, "
        . "color, material, style, target audience, key visible features, "
        . "size estimation. Return as JSON.";

    $analysis = wp_ai_client_prompt($prompt)
        ->with_image_url($image_url)
        ->using_model_preference('claude-sonnet-4-6', 'gpt-5.4')
        ->as_json_response([
            'product_name' => ['type' => 'string'],
            'category' => ['type' => 'string'],
            'description' => ['type' => 'string'],
            'features' => ['type' => 'array', 'items' => ['type' => 'string']],
            'color' => ['type' => 'string'],
            'material' => ['type' => 'string'],
            'style' => ['type' => 'string'],
            'target_audience' => ['type' => 'string'],
            'estimated_price_range' => ['type' => 'string'],
        ])
        ->generate_text();

    return json_decode($analysis, true);
}

// Fallback: OpenAI Vision API directly
$response = wp_remote_post($this->ai_endpoint, [
    'headers' => [
        'Authorization' => 'Bearer ' . getenv('OPENAI_API_KEY'),
        'Content-Type' => 'application/json',
    ],
    'body' => json_encode([
        'model' => 'gpt-4o',
        'messages' => [
            [
                'role' => 'user',
                'content' => [
                    ['type' => 'text', 'text' => 'Describe this product for e-commerce listing. Response format: JSON with product_name, category, description, features[], color, material, style, target_audience, estimated_price_range'],
                    ['type' => 'image_url', 'image_url' => ['url' => $image_url]],
                ],
            ],
        ],
        'response_format' => ['type' => 'json_object'],
    ]),
    'timeout' => 60,
]);

$body = json_decode(wp_remote_retrieve_body($response), true);
return json_decode($body['choices'][0]['message']['content'] ?? '{}', true);
}

/**
 * Generate complete product data (title, description, SEO, price, SKU)
 */
private function generate_product_data($image_analysis) {
    $existing_categories = get_terms([
        'taxonomy' => 'product_cat',
        'hide_empty' => false,
        'fields' => 'names',
    ]);

    // Generate SEO-optimized title
    $title_prompt = sprintf(
        "Create an SEO-optimized WooCommerce product title for: %s. "
        . "Category: %s. Include primary keyword. Max 70 chars.",
        $image_analysis['product_name'] ?? 'Product',
        $image_analysis['category'] ?? 'General'
    );

    $title = wp_ai_client_prompt($title_prompt)
        ->using_temperature(0.7)
        ->generate_text();

    // Generate long description with SEO
    $desc_prompt = sprintf(
        "Write a compelling WooCommerce product description (300-400 words) for: %s. "
        . "Features: %s. Material: %s. Style: %s. Target: %s. "
        . "Include: intro paragraph, features as bullet points, benefits to buyer, specs section, CTA. Use SEO keywords naturally.",
        $image_analysis['product_name'] ?? 'Product',
        implode(' ', $image_analysis['features'] ?? []),
        $image_analysis['material'] ?? 'N/A',
        $image_analysis['style'] ?? 'N/A',
        $image_analysis['target_audience'] ?? 'General'
    );

    $description = wp_ai_client_prompt($desc_prompt)
        ->using_temperature(0.8)
        ->generate_text();

    // Generate short description (excerpt)
    $short_desc = wp_ai_client_prompt(
        "Summarize this product description in 2-3 sentences for WooCommerce short description: {$description}"
    )
        ->using_temperature(0.6)
        ->generate_text();

    // Suggest price based on market research
    $price_prompt = sprintf(
        "Based on this product: %s (category: %s, features: %s, estimated range: %s), "
        . "suggest a competitive retail price in USD. Return ONLY the number (e.g., 29.99).",
        $image_analysis['product_name'] ?? 'Product',
        $image_analysis['category'] ?? 'General',
        implode(' ', $image_analysis['features'] ?? []),
        $image_analysis['estimated_price_range'] ?? 'medium'
    );

    $price = floatval(wp_ai_client_prompt($price_prompt)
        ->using_temperature(0.3)
        ->generate_text());

    // Generate SKU
    $sku = $this->generate_sku($image_analysis);

    // Suggest categories
    $cat_prompt = sprintf(
        "Which of these WooCommerce categories best fits this product: %s (%s)? "
        . "Available categories: %s. Return 1-2 category names, comma-separated.",
        $image_analysis['product_name'] ?? 'Product',
        $image_analysis['category'] ?? 'General',
        implode(' ', $existing_categories)
    );

    $categories = explode(' ', wp_ai_client_prompt($cat_prompt)
        ->using_temperature(0.3)
        ->generate_text());
    $categories = array_map('trim', $categories);

    // Generate tags
    $tag_prompt = sprintf(
        "Generate 5-10 SEO tags (comma-separated) for this product: %s. Features: %s.",
        $image_analysis['product_name'] ?? 'Product',
        implode(' ', $image_analysis['features'] ?? [])
    );

    $tags = explode(' ', wp_ai_client_prompt($tag_prompt)
        ->using_temperature(0.7)
        ->generate_text());
    $tags = array_map('trim', $tags);

    return [
        'title' => $title,
        'description' => $description,
        'short_description' => $short_desc,
        'price' => $price ? : 29.99,
    ];
}

```

```
        'sku' => $sku,
        'categories' => $categories,
        'tags' => $tags,
        'color' => $image_analysis['color'] ?? '',
        'material' => $image_analysis['material'] ?? '',
        'target_audience' => $image_analysis['target_audience'] ?? ''
    ];
}

/**
 * Generate unique SKU from product data
 */
private function generate_sku($analysis) {
    $category_prefix = strtoupper(substr($analysis['category'] ?? 'GEN', 0, 3));
    $color_code = strtoupper(substr($analysis['color'] ?? 'XX', 0, 2));
    $random = substr(uniqid(), -4);
    return "{$category_prefix}-{$color_code}-{$random}";
}

/**
 * Create WooCommerce product via REST API
 */
private function create_product($data, $attachment_id) {
    $product = new WC_Product_Simple();
    $product->set_name($data['title']);
    $product->set_description($data['description']);
    $product->set_short_description($data['short_description']);
    $product->set_regular_price($data['price']);
    $product->set_sku($data['sku']);
    $product->set_status('publish');
    $product->set_catalog_visibility('visible');
    $product->set_image_id($attachment_id);

    // Stock management
    $product->set_manage_stock(true);
    $product->set_stock_quantity(100); // Default initial stock
    $product->set_stock_status('instock');

    // Attributes
    if (!empty($data['color'])) {
        $attribute = new WC_Product_Attribute();
        $attribute->set_name('Color');
        $attribute->set_options([$data['color']]);
        $attribute->set_visible(true);
        $attribute->set_variation(false);
        $product->set_attributes([$attribute]);
    }

    // Categories
    if (!empty($data['categories'])) {
        $cat_ids = [];
        foreach ($data['categories'] as $cat_name) {
            $term = term_exists($cat_name, 'product_cat');
            if (!$term) {
                $term = wp_insert_term($cat_name, 'product_cat');
            }
            if (!is_wp_error($term)) {
                $cat_ids[] = is_array($term) ? $term['term_id'] : $term;
            }
        }
        $product->set_category_ids($cat_ids);
    }

    // Tags
    if (!empty($data['tags'])) {
        $product->set_tag_ids(array_map(function($tag) {
            $term = term_exists($tag, 'product_tag');
            if (!$term) $term = wp_insert_term($tag, 'product_tag');
            return is_array($term) ? $term['term_id'] : $term;
        }, $data['tags']));
    }

    $product_id = $product->save();

    // Add AI-generated alt text to image
    update_post_meta($attachment_id, '_wp_attachment_image_alt', $data['title']);

    return $product_id;
}

// Run the importer
$creator = new AI_Bulk_Product_Creator();
$creator->scan_and_create();
```

Usage:

```
# 1. Place product images in wp-content/uploads/bulk-import/
# 2. Run the creator
wp eval-file ai-bulk-product-creator.php

# 3. Or set up a cron job for auto-processing
wp cron event schedule ai_bulk_product_import --hourly
```

4.3 Existing AI Product Generator Projects

Project	Stars	What It Does	URL
AI Auto Uploader for WC	■1	Scans directories for images, extracts titles, generates AI descriptions, detects prices from filenames, Rank Math SEO	GitHub
SynapseCommerce AI	■1	Multi-agent AI pipeline: discovers trending products, analyzes competitor pricing, generates SEO content, publishes to WooCommerce	GitHub
woocommerce-ai-image-generator	■0	Auto-generates product images for WC products missing images via AI image service	GitHub
AI-Powered Product Detail Generator	■0	Reads product images, auto-generates short/long descriptions, pricing, outputs JSON for WooCommerce/Amazon/Meesho	GitHub

4.4 AI Product Categorization & Tagging

```
/**
 * AI categorization for existing products missing categories
 * Run: wp eval-file categorize-products.php
 */
function ai_categorize_all_products() {
    $products = wc_get_products(['limit' => -1, 'status' => 'publish']);
    $categories = get_terms(['taxonomy' => 'product_cat', 'hide_empty' => false, 'fields' => 'names']);

    foreach ($products as $product) {
        // Skip already categorized
        if (!empty($product->get_category_ids())) continue;
```

```

$name = $product->get_name();
$desc = $product->get_short_description() ?: $product->get_description();

$prompt = "Product: \"{$name}\". Description: {$desc}. "
    . "Available categories: " . implode(' ', $categories) . ". "
    . "Which 1-2 categories? Return JSON: {\"categories\":[\"cat1\\\", \"cat2\\\"]}";

$result = wp_ai_client_prompt($prompt)
    ->as_json_response(['categories' => ['type' => 'array']])
    ->generate_text();

$data = json_decode($result, true);

if (!empty($data['categories'])) {
    $cat_ids = [];
    foreach ($data['categories'] as $cat_name) {
        $term = term_exists(trim($cat_name), 'product_cat');
        if ($term) {
            $cat_ids[] = is_array($term) ? $term['term_id'] : $term;
        }
    }
    $product->set_category_ids($cat_ids);
    $product->save();
    WP_CLI::log("Categorized: {$name} → " . implode(' ', $data['categories']));
}
}
}

```

5. AI-Powered Store Automation Workflows

5.1 AI Inventory Forecasting & Reorder Alerts

shopops-mcp (🟡): AI e-commerce operations manager via MCP. Inventory forecasting, pricing optimization, RFM customer segmentation, order anomaly detection for Shopify & WooCommerce.

[GitHub](#)

Custom Implementation:

```

/**
 * AI inventory reorder prediction
 */
function ai_reorder_prediction() {
    $products = wc_get_products(['limit' => -1, 'manage_stock' => true]);

    $predictions = [];

    foreach ($products as $product) {
        $stock = $product->get_stock_quantity();
        if ($stock === null) continue;

        // Calculate daily sales rate (last 90 days)
        $orders = wc_get_orders([
            'limit' => -1,
            'status' => ['completed', 'processing'],
            'date_created' => '>=' . date('Y-m-d', strtotime('-90 days')),
        ]);

        $units_sold = 0;
        foreach ($orders as $order) {
            foreach ($order->get_items() as $item) {
                if ($item->get_product_id() == $product->get_id()) {
                    $units_sold += $item->get_quantity();
                }
            }
        }

        $daily_rate = $units_sold / 90;
        $days_remaining = $daily_rate > 0 ? $stock / $daily_rate : 999;

        $predictions[] = [
            'product_id' => $product->get_id(),
            'product_name' => $product->get_name(),
            'current_stock' => $stock,
            'daily_sales_rate' => round($daily_rate, 2),
            'days_remaining' => round($days_remaining, 0),
            'urgency' => $days_remaining < 14 ? 'HIGH' : ($days_remaining < 30 ? 'MEDIUM' : 'LOW'),
        ];
    }

    // Sort by urgency
    usort($predictions, fn($a, $b) => $a['days_remaining'] <=> $b['days_remaining']);

    // AI analysis of predictions
    $ai_analysis = wp_ai_client_prompt(
        "Analyze this WooCommerce inventory forecast and recommend reorder quantities. "
        . "Consider seasonality, lead time, and safety stock. Data: " . json_encode($predictions)
    )->generate_text();

    // Send admin email with AI recommendations
    $admin_email = get_option('admin_email');
    wp_mail($admin_email, 'AI Inventory Reorder Report', $ai_analysis);

    return $predictions;
}

```

5.2 AI Pricing Optimization

Dynamic Pricing Pattern:

```

/**
 * AI competitor price monitoring and dynamic pricing
 */
function ai_dynamic_pricing() {
    // Fetch product prices
    $products = wc_get_products(['limit' => 50, 'status' => 'publish', 'orderby' => 'date']);

    $pricing_data = [];
    foreach ($products as $product) {
        $pricing_data[] = [
            'id' => $product->get_id(),
            'name' => $product->get_name(),
            'price' => $product->get_regular_price(),
            'sale_price' => $product->get_sale_price(),
            'sales_last_30d' => get_product_sales_count($product->get_id(), 30),
            'views_last_30d' => get_product_views($product->get_id(), 30),
            'conversion_rate' => get_product_conversion_rate($product->get_id(), 30),
            'margin' => get_product_margin($product->get_id()),
        ];
    }

    // AI pricing recommendations
}

```

```

$recommendations = wp_ai_client_prompt(
    "Analyze these WooCommerce products and recommend price adjustments. "
    . "For each: suggest new regular price, sale price (if relevant), and reasoning. "
    . "Consider conversion rate, sales velocity, and margin. "
    . "Never suggest prices below cost. Return JSON array. "
    . "Data: " . json_encode($pricing_data)
)->as_json_response([
    'adjustments' => ['type' => 'array', 'items' => [
        'type' => 'object',
        'properties' => [
            'product_id' => ['type' => 'integer'],
            'new_price' => ['type' => 'number'],
            'new_sale_price' => ['type' => 'number'],
            'reasoning' => ['type' => 'string'],
        ],
    ]],
]);
$recommendations->generate_text();

return json_decode($recommendations, true);
}

```

5.3 AI Customer Segmentation (RFM)

```

/**
 * AI RFM (Recency, Frequency, Monetary) customer segmentation
 */
function ai_customer_segmentation() {
    $customers = get_users(['role' => 'customer']);
    $segments = [];

    foreach ($customers as $customer) {
        $orders = wc_get_orders([
            'customer_id' => $customer->ID,
            'limit' => -1,
            'status' => ['completed'],
        ]);

        if (empty($orders)) continue;

        $last_order = $orders[0]->get_date_created()->getTimestamp();
        $days_since_last = (time() - $last_order) / DAY_IN_SECONDS;

        $total_spent = array_sum(array_map(fn($o) => $o->get_total(), $orders));

        $segments[] = [
            'customer_id' => $customer->ID,
            'name' => $customer->display_name,
            'email' => $customer->user_email,
            'order_count' => count($orders),
            'total_spent' => $total_spent,
            'days_since_last' => round($days_since_last, 0),
            'avg_order_value' => round($total_spent / count($orders), 2),
        ];
    }

    // AI segmentation
    $ai_segments = wp_ai_client_prompt(
        "Segment these WooCommerce customers using RFM analysis into: "
        . "Champions, Loyal Customers, At Risk, Lost, New Customers. "
        . "For each customer, assign a segment and suggest a personalized marketing action. "
        . "Return JSON. Data: " . json_encode($segments)
    )->as_json_response([
        'segments' => ['type' => 'array'],
    ])->generate_text();

    $result = json_decode($ai_segments, true);

    // Store segments as user meta for email targeting
    foreach ($result['segments'] ?? [] as $seg) {
        update_user_meta($seg['customer_id'], 'ai_customer_segment', $seg['segment']);
        update_user_meta($seg['customer_id'], 'ai_marketing_action', $seg['marketing_action'] ?? '');
    }

    return $result;
}

```

5.4 AI Abandoned Cart Recovery

```

/**
 * AI-powered abandoned cart recovery emails
 * Uses personalized product recommendations and urgency
 */
function ai_abandoned_cart_campaign() {
    // Get abandoned carts (last 24 hours)
    $carts = get_posts([
        'post_type' => 'shop_order',
        'post_status' => 'wc-pending',
        'posts_per_page' => -1,
        'date_query' => [
            ['after' => '24 hours ago'],
        ],
    ]);

    foreach ($carts as $cart_post) {
        $order = wc_get_order($cart_post->ID);
        if (!$order || $order->get_total() == 0) continue;

        $customer_email = $order->get_billing_email();
        $items = [];
        foreach ($order->get_items() as $item) {
            $product = $item->get_product();
            if ($product) {
                $items[] = [
                    'name' => $product->get_name(),
                    'price' => $product->get_price(),
                    'image' => wp_get_attachment_url($product->get_image_id()),
                ];
            }
        }

        // AI-generated personalized recovery email
        $email_body = wp_ai_client_prompt(
            "Write a personalized abandoned cart recovery email for a customer who left these items: "
            . json_encode($items) . ". Cart total: $" . $order->get_total() . ". "
            . "Tone: friendly but not pushy. Include: item names, urgency (low stock if applicable), "
            . "a small discount incentive (5-10%), and a clear CTA to return to cart. "
            . "The cart URL is: " . wc_get_cart_url()
        )->using_temperature(0.8)->generate_text();

        // Generate subject line
        $subject = wp_ai_client_prompt(
            "Write a compelling email subject line (max 50 chars) for an abandoned cart "
            . "recovery email. Items: " . json_encode(array_column($items, 'name'))
        )->using_temperature(0.9)->generate_text();

        // Send email
        $headers = ['Content-Type: text/html; charset=UTF-8'];
        wp_mail($customer_email, $subject, $email_body, $headers);
    }
}

```

```

    // Log the recovery attempt
    $order->add_order_note("AI abandoned cart email sent: {$subject}");
  }
}

```

5.5 AI Product Recommendation Engine

```

/**
 * AI product recommendations based on browsing history + collaborative filtering
 */
function ai_product_recommendations($current_product_id) {
    $current_product = wc_get_product($current_product_id);
    if (!$current_product) return [];

    $category_ids = $current_product->get_category_ids();
    $tag_ids = $current_product->get_tag_ids();
    $price = $current_product->get_price();

    // Get products in same categories/tags
    $related = wc_get_products([
        'limit' => 50,
        'category' => $category_ids,
        'tag' => $tag_ids,
        'exclude' => [$current_product_id],
        'status' => 'publish',
    ]);

    $candidates = [];
    foreach ($related as $product) {
        $candidates[] = [
            'id' => $product->get_id(),
            'name' => $product->get_name(),
            'price' => $product->get_price(),
            'categories' => wp_get_post_terms($product->get_id(), 'product_cat', ['fields' => 'names']),
            'sales_30d' => get_product_sales_count($product->get_id(), 30),
            'avg_rating' => $product->get_average_rating(),
            'review_count' => $product->get_review_count(),
        ];
    }

    // AI-powered recommendation ranking
    $recommendations = wp_ai_client_prompt(
        "Given a product at price \${$price} and these related products, "
        . "Recommend the top 4 that are most complementary (cross-sell) or similar (upsell). "
        . "Consider: price range, complementary categories, review scores, popularity. "
        . "Return JSON array of product IDs in ranked order. "
        . "Related products: " . json_encode($candidates)
    )->using_temperature(0.4)->generate_text();

    $ranked_ids = json_decode($recommendations, true);
    $ranked_ids = array_slice($ranked_ids, 0, 4);

    // Fetch full product data for display
    return array_map(fn($id) => wc_get_product($id), $ranked_ids);
}

```

5.6 AI Review & Feedback Analysis

```

/**
 * AI review sentiment analysis and automated response
 */
function ai_review_analysis() {
    $reviews = get_comments([
        'post_type' => 'product',
        'status' => 'approve',
        'number' => 100,
    ]);

    $analysis = [];
    foreach ($reviews as $review) {
        $analysis[] = [
            'review_id' => $review->comment_ID,
            'product_id' => $review->comment_post_ID,
            'rating' => get_comment_meta($review->comment_ID, 'rating', true),
            'content' => $review->comment_content,
            'author' => $review->comment_author,
        ];
    }

    // AI sentiment & trend analysis
    $insights = wp_ai_client_prompt(
        "Analyze these WooCommerce product reviews. Identify: "
        . "1. Overall sentiment (positive/negative/mixed) "
        . "2. Top 5 recurring themes/issues "
        . "3. Top 5 praised features "
        . "4. Products with most complaints "
        . "5. Actionable recommendations for store improvement. "
        . "Return JSON. Reviews: " . json_encode($analysis)
    )->generate_text();

    // Log insights for admin dashboard
    update_option('ai_review_insights', $insights);
    update_option('ai_review_insights_date', current_time('mysql'));

    // Auto-generate review responses for low-rated reviews without store response
    foreach ($analysis as $r) {
        if ($r['rating'] <= 2) {
            $has_response = get_comments([
                'parent' => $r['review_id'],
                'count' => true,
            ]);

            if ($has_response == 0) {
                $response = wp_ai_client_prompt(
                    "Write a professional, empathetic response to this negative product review "
                    . "({$r['rating']} stars). Address concerns, offer solution, maintain brand voice. "
                    . "Review: {$r['content']}"
                )->generate_text();

                // Store as draft for admin approval
                wp_insert_comment([
                    'comment_post_ID' => $r['product_id'],
                    'comment_parent' => $r['review_id'],
                    'comment_content' => $response,
                    'comment_approved' => 0, // Draft - awaits admin approval
                    'comment_type' => 'comment',
                ]);
            }
        }
    }

    return json_decode($insights, true);
}

```

6. Agentic Commerce Protocols

6.1 The Four Protocols

Protocol	Backers	Problem Solved	Where It Runs
UCP	Google + Shopify + 20+ retailers	Full lifecycle: discovery, cart, identity, checkout, orders	Google AI Mode, Gemini
ACP	Stripe + OpenAI	Agent-driven checkout	ChatGPT (Apps, select retailers)
AP2	Google + 60+ payment partners	Proving human authorized agent's payment	Payment layer for UCP/ACP
MCP	Anthropic (open standard)	Connecting agents to tools/data	Claude, ChatGPT, any MCP client

6.2 WooCommerce Agentic Commerce Plugins

Agentic Commerce for WooCommerce (Free, wp.org): - Makes products discoverable and buyable by AI assistants - Agent-readable catalog, JSON-LD schema, `llms.txt` support, cart deep links - [wp.org/plugins/agentic-commerce-for-woocommerce/](#)

UCP Checkout for WooCommerce (Free, GitHub): - Enable AI agents (ChatGPT/Gemini/Claude) to discover and purchase products via Universal Commerce Protocol - [GitHub](#)

6.3 Agentic Commerce Skills/Plugins for AI Coders

OrcaQubits/agentic-commerce-skills-plugins (134): - 14 commerce plugins for Claude Code, Cursor, Codex, Gemini, OpenClaw - Includes: WooCommerce expert agent (20 skills), UCP agent, ACP agent, AP2 agent, MPP agent, WebMCP agent - Install: `npm skills add orcaqubits/agentic-commerce-skills-plugins` - [GitHub](#)

WooCommerce Expert Agent (within above): Full WooCommerce + PHP 8.x knowledge with live doc fetching. Covers: plugin dev, hooks/filters, data stores, HPOS, REST API, checkout blocks, payment gateways, shipping methods, catalog, testing, deployment.

7. MCP Servers for E-Commerce

7.1 Complete MCP Server Directory

Server	Stars	Platform	Type	URL
techspawn/woocommerce-mcp-server	96	WooCommerce	Self-hosted (Node.js)	GitHub
MentionNetwork/awesome-agentic-commerce	89	Cross-platform	Resource list	GitHub
OrcaQubits/agentic-commerce-skills-plugins	34	Cross-platform	Claude Code plugins	GitHub
tropk-ai/mcp-for-wordpress	27	WordPress/WooCommerce	OAuth MCP (500+ tools)	GitHub
iOSDevSK/mcp-for-woocommerce	15	WooCommerce	WP plugin (STDIO/HTTP)	GitHub
paypal/paypal-mcp-server	11	PayPal	Official (npm)	GitHub
iannuttall/mcp-boilerplate	1K	Stripe	Cloudflare MCP (+Stripe)	GitHub
shopanaio/carrier-api	6	Shipping carriers	MCP server	GitHub
Stripe MCP Server (Official)	—	Stripe	Remote (docs.stripe.com)	docs.stripe.com/mcp
woocommerce/wc-mcp-ability	5	WooCommerce	Official demo plugin	GitHub
DSers Official MCP	3	Dropshipping	Remote (OAuth)	GitHub Docs
wppoland/woocommerce-mcp	2	WooCommerce	Read-only REST	GitHub
shopops-mcp	0	Shopify/WC	AI ops (forecast/price/segmentation)	GitHub
VOC AI MCP	—	Amazon/WC reviews	Sentiment analysis	voc.ai
Stripe AI Toolkit	—	Stripe	Official AI tools	GitHub

7.2 Top WooCommerce MCP Server — Deep Dive

techspawn/woocommerce-mcp-server (96) — The most comprehensive WooCommerce MCP.

Configuration:

```
{
  "mcpServers": {
    "woocommerce": {
      "command": "node",
      "args": ["path/to/build/index.js"],
      "env": {
        "WORDPRESS_SITE_URL": "https://yoursite.com",
        "WOOCOMMERCE_CONSUMER_KEY": "ck_XXX",
        "WOOCOMMERCE_CONSUMER_SECRET": "cs_XXX",
        "WORDPRESS_USERNAME": "admin",
        "WORDPRESS_PASSWORD": "app_password_XXX"
      }
    }
  }
}
```

Available Tools (partial list): - Products: `get_products`, `get_product`, `create_product`, `update_product`, `delete_product`, `get_product_meta`, `update_product_meta` - Orders: `get_orders`, `get_order`, `create_order`, `update_order`, `delete_order` - Customers: `get_customers`, `get_customer`, `create_customer`, `update_customer` - Coupons: `get_coupons`, `create_coupon`, `update_coupon`, `delete_coupon` - Shipping: `get_shipping_zones`, `get_shipping_methods`, `create_shipping_zone`, `update_shipping_zone` - Taxes: `get_tax_rates`, `get_tax_classes`, `create_tax_rate` - Reports: `get_sales_report`, `get_products_report`, `get_orders_report`, `get_customers_report` - Settings: `get_store_settings`, `update_store_settings` - Webhooks: `get_webhooks`, `create_webhook`, `delete_webhook` - WP Content: `create_post`, `get_posts`, `update_post`, `get_post_meta`

8. n8n WooCommerce Automation Templates

8.1 n8n WooCommerce Node Reference

Trigger nodes: - WooCommerce Trigger: New order, order status changed, new customer, new product review, low stock

Action nodes: - WooCommerce: Create/update/delete product, order, customer, coupon - WooCommerce: Get all products, orders, customers, reports

8.2 Pre-built Workflow Templates

Workflow	What It Does	Source
Abandoned Cart Recovery	Detects abandoned carts → verifies payment status → AI-personalized recovery email → Slack alert	GitHub
Review Analysis	Analyzes WC reviews with GPT-4 → stores in Airtable → Slack sentiment summaries	GitHub

VIP Customer Detection	Analyzes order history → calculates LTV + frequency → assigns VIP tiers → Slack alerts	GitHub
Failed Order Monitoring	Polls for failed orders → dedupes via Airtable → AI-generated Slack alerts	GitHub
Return Surge Detection	Monitors return activity → detects spikes → Slack alerts + Airtable logging	GitHub
Missing Address Fix	Detects missing address orders → AI extracts from context → SMS/email notification	GitHub
Dropshipping Pipeline	Google Trends → AliExpress → AI content → WooCommerce publish	GitHub
Customer Support Agent	WC API + DHL tracking + OpenAI GPT-4 → real-time order/shipping support	GitHub
Category Sales Analysis	Compares WC sales across periods → classifies trends → Airtable + Slack reports	GitHub
E-Commerce Auto-Manager	WC + Shopify: manage orders, cancel orders, auto-invoice via email	GitHub

8.3 Example: AI Abandoned Cart n8n Workflow

```
[WooCommerce Trigger: New Pending Order]
↓
[IF: Order date > 1 hour ago → Continue]
↓
[HTTP Request: GET /wc/v3/orders/{id} – verify still pending]
↓
[OpenAI Node: Generate personalized email body]
  - Input: cart items, customer name, total, product images
  - Prompt: "Write abandoned cart email..."
↓
[Gmail Node: Send recovery email]
↓
[Slack Node: Notify team "Abandoned cart recovery sent"]
↓
[WooCommerce: Add order note "Recovery email sent"]
```

9. Store Setup Automation & Bootstrap

9.1 One-Command Store Bootstrap (WP-CLI)

```
#!/bin/bash
# woocommerce-bootstrap.sh – One-command WooCommerce store setup
# Usage: bash woocommerce-bootstrap.sh sitename.com "My Store" "admin@example.com"

SITE_URL=$1
STORE_NAME=$2
ADMIN_EMAIL=$3
CURRENCY=${4:-USD}
COUNTRY=${5:-US}

echo "■ Bootstrapping WooCommerce store: $STORE_NAME"

# 1. WP Core setup
wp core install \
  --url="$SITE_URL" \
  --title="$STORE_NAME" \
  --admin_user="admin" \
  --admin_email="$ADMIN_EMAIL" \
  --skip-email

# 2. Configure permalinks
wp rewrite structure '/%postname%/'
wp rewrite flush

# 3. Install + activate WooCommerce
wp plugin install woocommerce --activate

# 4. Create default WooCommerce pages
wp wc tool run install_pages

# 5. Configure WC settings
wp option update woocommerce_store_address "123 Commerce St"
wp option update woocommerce_store_city "New York"
wp option update woocommerce_default_country "${COUNTRY}:NY"
wp option update woocommerce_currency "$CURRENCY"
wp option update woocommerce_weight_unit "lbs"
wp option update woocommerce_dimension_unit "in"
wp option update woocommerce_enable_guest_checkout "yes"
wp option update woocommerce_enable_signup_and_login_from_checkout "yes"
wp option update woocommerce_enable_myaccount_registration "yes"

# 6. Install payment gateways
wp plugin install woocommerce-gateway-stripe --activate
wp plugin install woocommerce-paypal-payments --activate
wp plugin install woocommerce-services --activate

# 7. Install essential plugins
wp plugin install wordfence --activate
wp plugin install wordpress-seo --activate
wp plugin install mailchimp-for-woocommerce --activate
wp plugin install wp-super-cache --activate

# 8. Enable HPOS
wp wc cot enable

# 9. Set up tax (automated)
wp option update wc_connect_taxes_enabled "yes"

# 10. Create essential pages
wp post create --post_type=page --post_title="About Us" --post_status=publish
wp post create --post_type=page --post_title="Contact" --post_status=publish
wp post create --post_type=page --post_title="Shipping Policy" --post_status=publish
wp post create --post_type=page --post_title="Returns & Refunds" --post_status=publish
wp post create --post_type=page --post_title="Privacy Policy" --post_status=publish
wp post create --post_type=page --post_title="Terms of Service" --post_status=publish
wp post create --post_type=page --post_title="FAQ" --post_status=publish

# 11. Create basic shipping zones
wp wc shipping_zone create --name="Domestic" --order=1
wp wc shipping_zone_method create 1 --method_id="free_shipping" \
  --method_title="Free Shipping (orders over \$50)" \
  --settings='{ "min_amount": "50" }'

echo "■ WooCommerce store bootstrapped! Visit: $SITE_URL/wp-admin"
echo "■ Next: Configure Stripe keys, PayPal keys, shipping rates"
```

9.2 AI-Generated Store Policies

```
/**
 * Auto-generate store policies with AI
 * Run: wp eval-file generate-policies.php
 */
function ai_generate_store_policies($store_name, $store_type) {
```

```

$policies = [
    'returns_refunds' => [
        'title' => 'Returns & Refunds Policy',
        'prompt' => "Write a professional Returns & Refunds Policy for {$store_name}, "
            . "a {$store_type} e-commerce store. Include: return window (30 days), "
            . "condition requirements, refund process, exchanges, non-returnable items, "
            . "return shipping responsibility. 500-800 words. Friendly but clear tone."
    ],
    'privacy' => [
        'title' => 'Privacy Policy',
        'prompt' => "Write a GDPR-compliant Privacy Policy for {$store_name}. "
            . "Include: data collected (name, email, address, payment), "
            . "how data is used, third-party sharing (payment processors, shipping), "
            . "cookies, user rights, data retention, contact info. 800-1200 words."
    ],
    'terms' => [
        'title' => 'Terms of Service',
        'prompt' => "Write Terms of Service for {$store_name}. Include: "
            . "account registration, product descriptions accuracy, "
            . "pricing, payment terms, shipping terms, "
            . "limitation of liability, intellectual property, "
            . "governing law, dispute resolution. 800-1200 words."
    ],
    'shipping' => [
        'title' => 'Shipping Policy',
        'prompt' => "Write a Shipping Policy for {$store_name}. Include: "
            . "processing time (1-2 business days), shipping methods "
            . "(standard/express/international), delivery timeframes, "
            . "shipping costs, tracking info, international customs, "
            . "lost/damaged packages policy. 400-600 words."
    ],
];

foreach ($policies as $slug => $policy) {
    $content = wp_ai_client_prompt($policy['prompt'])
        ->using_temperature(0.6)
        ->generate_text();

    // Check if page already exists
    $existing = get_page_by_title($policy['title'], OBJECT, 'page');

    if ($existing) {
        wp_update_post([
            'ID' => $existing->ID,
            'post_content' => wp_kses_post($content),
        ]);
        WP_CLI::success("Updated: {$policy['title']}");
    } else {
        wp_insert_post([
            'post_title' => $policy['title'],
            'post_content' => wp_kses_post($content),
            'post_status' => 'publish',
            'post_type' => 'page',
            'post_name' => $slug,
        ]);
        WP_CLI::success("Created: {$policy['title']}");
    }
}

// Usage
ai_generate_store_policies('My Store', 'fashion');

```

9.3 AI-Generated Product Category Structure

```

/**
 * AI generates optimal product category hierarchy
 */
function ai_generate_category_tree($store_type, $store_name) {
    $prompt = "Generate a logical product category hierarchy for a {$store_type} "
        . "e-commerce store called '{$store_name}'. "
        . "Return as a JSON tree with parent categories and 3-5 subcategories each. "
        . "Maximum depth: 3 levels. Example format: "
        . '["(name":"Clothing","children":[{"name":"Tops","children":[]}])]';

    $tree = wp_ai_client_prompt($prompt)
        ->using_temperature(0.5)
        ->generate_text();

    $categories = json_decode($tree, true);

    function create_category_tree($items, $parent_id = 0) {
        foreach ($items as $item) {
            $term = wp_insert_term($item['name'], 'product_cat', [
                'parent' => $parent_id,
                'slug' => sanitize_title($item['name']),
            ]);

            if (!is_wp_error($term)) {
                $term_id = $term['term_id'];
                $children = $item['children'] ?? [];
                if (!empty($children)) {
                    create_category_tree($children, $term_id);
                }
            }
        }
    }

    create_category_tree($categories);
    WP_CLI::success("Category tree created for: {$store_type}");
}

```

9.4 AI-Generated Email Templates

```

/**
 * AI generates all WooCommerce transactional email templates
 */
function ai_generate_email_templates($store_name, $brand_voice) {
    $emails = [
        'new_order_admin' => [
            'subject' => "New Order #[order_id] - [customer_name]",
            'prompt' => "Write a concise admin notification email for a new order. Store: {$store_name}. Tone: {$brand_voice}."
        ],
        'customer_completed_order' => [
            'subject' => "Your {$store_name} order #[order_id] is complete! 🎉",
            'prompt' => "Write a warm, celebratory order completion email. Store: {$store_name}. Include: thank you, order recap, shipping tracking, review request. Tone: {$brand_voice}."
        ],
        'customer_processing_order' => [
            'subject' => "We're working on your {$store_name} order #[order_id]",
            'prompt' => "Write a reassuring order processing email. Store: {$store_name}. Include: order confirmation, processing time, what's next. Tone: {$brand_voice}."
        ],
        'customer_on_hold_order' => [
            'subject' => "Your {$store_name} order #[order_id] is on hold",
            'prompt' => "Write an order on-hold notification. Store: {$store_name}. Explain payment pending reason, next steps. Tone: {$brand_voice}."
        ],
        'customer_new_account' => [
            'subject' => "Welcome to {$store_name}! 🎊",
            'prompt' => "Write a warm welcome email for new account. Store: {$store_name}. Include: benefits of account, quick links, support contact. Tone: {$brand_voice}."
        ],
    ],
}

```

```
'customer_reset_password' => [
    'subject' => "Password reset for your {$store_name} account",
    'prompt' => "Write a password reset email. Store: {$store_name}. Clear instructions, security note, support contact. Tone: {$brand_voice}."
],
];

foreach ($emails as $slug => $email) {
    $body = wp_ai_client_prompt(
        "Write a complete HTML email body (with inline CSS) for: {$email['prompt']}. "
        . "Include: store logo placeholder, personalized greeting, main message, "
        . "Clear CTA button, footer with contact/social links. "
        . "Email-safe HTML only (table-based layout). Max width 600px. "
        . "Use WC email placeholders: {site_title}, {order_number}, {customer_name}, "
        . "{order_date}, {billing_address}, {shipping_address}, {items_table}, {order_total}."
    )->using_temperature(0.7)->generate_text();

    // Save as WooCommerce email template override
    $template_path = get_stylesheet_directory() . "/woocommerce/emails/{$slug}.php";
    file_put_contents($template_path, $body);
    WP_CLI::success("Created email template: {$slug}");
}
}
```

9.5 AI A/B Testing for Product Pages

```
/**
 * AI generates A/B test variants for product pages
 */
function ai_generate_ab_variants($product_id) {
    $product = wc_get_product($product_id);
    if (!$product) return;

    $current = [
        'title' => $product->get_name(),
        'description' => $product->get_description(),
        'short_desc' => $product->get_short_description(),
        'price' => $product->get_price(),
    ];

    // Generate 3 variant strategies
    $variants = wp_ai_client_prompt(
        "Create 3 product page variants for A/B testing this product: "
        . json_encode($current) . " "
        . "Variant A: Benefit-focused (lead with benefits, not features). "
        . "Variant B: Social-proof focused (include testimonials, stats, ratings). "
        . "Variant C: Urgency-focused (limited stock, time-sensitive offer). "
        . "For each variant, provide: title, short description, key selling points, CTA. "
        . "Return JSON."
    )->as_json_response([
        'variants' => ['type' => 'array', 'items' => ['type' => 'object']],
    ]->generate_text());

    $data = json_decode($variants, true);

    // Store variants for A/B testing framework
    update_post_meta($product_id, '_ab_test_variants', $data['variants'] ?? []);
    update_post_meta($product_id, '_ab_test_status', 'ready');

    return $data;
}
```

10. Reference Index

Payment Gateways

Resource	URL
Stripe WC Gateway (GitHub)	https://github.com/woocommerce/woocommerce-gateway-stripe
Stripe WC Gateway (WP.org)	https://wordpress.org/plugins/woocommerce-gateway-stripe/
Stripe MCP Server (Official)	https://docs.stripe.com/mcp
Stripe AI Toolkit	https://github.com/stripe/ai
Stripe Developer Docs	https://stripe.com/docs/api
PayPal Payments (WP.org)	https://wordpress.org/plugins/woocommerce-paypal-payments/
PayPal MCP Server (Official)	https://github.com/paypal/paypal-mcp-server
PayPal Agent Toolkit	https://github.com/paypal/agent-toolkit
PayPal Developer	https://developer.paypal.com
WooPayments (Official)	https://woocommerce.com/products/woopayments/
Square for WC (WP.org)	https://wordpress.org/plugins/woocommerce-square/
Braintree for WC	https://wordpress.org/plugins/woocommerce-gateway-paypal-powered-by-braintree/

Shipping

Resource	URL
WC Shipping & Tax (GitHub)	https://github.com/Automattic/woocommerce-services
WC Shipping & Tax (WP.org)	https://wordpress.org/plugins/woocommerce-services/
Shippo API Docs	https://goshippo.com/docs
EasyPost API Docs	https://www.easypost.com/docs
ShipEngine API Docs	https://www.shipengine.com/docs
DSers MCP Server	https://github.com/dsers/dsers-mcp-server
Carrier API MCP	https://github.com/shopanaio/carrier-api

Tax

Resource	URL
TaxJar WC Plugin	https://github.com/taxjar/taxjar-woocommerce-plugin
TaxJar API Docs	https://developers.taxjar.com
Avalara Developer	https://developer.avalara.com
WooCommerce Tax Setup	https://woocommerce.com/document/setting-up-taxes-in-woocommerce/

MCP Servers

Resource	URL
----------	-----

WooCommerce MCP Server (🟢96)	https://github.com/techspawn/woocommerce-mcp-server
WooCommerce MCP (WP plugin, 🟢15)	https://github.com/iOSDevSK/mcp-for-woocommerce
WordPress MCP (500+ tools)	https://github.com/tropk-ai/mcp-for-wordpress
Awesome Agentic Commerce	https://github.com/MentionNetwork/awesome-agentic-commerce
Agentic Commerce Plugins	https://github.com/OrcaQubits/agentic-commerce-skills-plugins
ShopOps MCP	https://github.com/enzoemir1/shopops-mcp
WooCommerce MCP Ability (Official)	https://github.com/woocommerce/wc-mcp-ability

AI Product Creation

Resource	URL
AI Auto Uploader (WC)	https://github.com/Abdelrahman099/AI-Auto-Uploader-for-WooCommerce
SynapseCommerce AI	https://github.com/tayyabalitech/synapse-commerce-ai
AI Product Detail Generator	https://github.com/gitdsiuvn/ai_powerred_ecommerce_product_detail_generator
WC AI Image Generator	https://github.com/aamirshabbir/woocommerce-ai-image-generator

n8n Workflows

Resource	URL
Abandoned Cart Recovery	https://github.com/weblineindia/n8n-Recover-abandoned-WooCommerce-carts-using-OpenAI-GPT-4.1-mini-Gmail-and-Slack
Product Review Analysis	https://github.com/weblineindia/n8n-Analyze-WooCommerce-product-reviews-with-GPT-4-Airtable-Slack-summaries
VIP Customer Detection	https://github.com/weblineindia/n8n-Identify-and-notify-WooCommerce-VIP-customers-with-Airtable-and-Slack
Failed Order Monitoring	https://github.com/weblineindia/n8n-Log-failed-WooCommerce-orders-to-Airtable-and-send-OpenAI-powered-Slack-alerts
Dropshipping Automation	https://github.com/a-nayem/Dropshipping-automation
Customer Support Agent	https://github.com/haseebnaeem94/WooCommerce-Customer-Support-AI-Agent

Agentic Commerce Protocols

Resource	URL
UCP Specification	https://github.com/Universal-Commerce-Protocol/ucp
ACP Specification	https://github.com/agentic-commerce-protocol/agentic-commerce-protocol
AP2 Specification	https://github.com/google-agentic-commerce/AP2
MCP Protocol	https://modelcontextprotocol.io
Agentic Commerce for WC	https://wordpress.org/plugins/agentic-commerce-for-woocommerce/
UCP Checkout for WC	https://github.com/boxybird/ucp-checkout-for-woocommerce

Dropshipping

Resource	URL
DSers MCP Server	https://mcp.dsers.com/dropshipping/mcp
Dropshipping Automation (n8n)	https://github.com/a-nayem/Dropshipping-automation
Dropshipzone WC Plugin	https://github.com/shauncuier/dropshipzone
Dropshipping WP Platform	https://github.com/ramphor/dropshipping

WooCommerce E-Commerce Ecosystem Playbook — v1.0 | Compiled July 17, 2026 Companion to [woocommerce-ai-dev-playbook.md](#) — Core WP/WC dev, Divi 5, and security covered there.

■ PART C: WOOCOMMERCE OFFICIAL AI INTEGRATION (MCP + Agent Skills)

2. WOOCOMMERCE DEVELOPER AI DOCS (primary source)

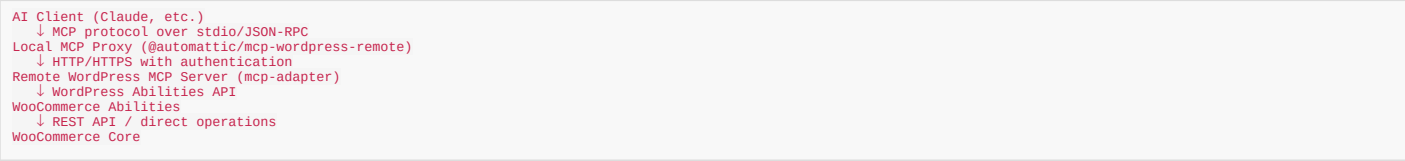
URL: <https://developer.woocommerce.com/docs/getting-started/ai/>

2.1 Native MCP (Model Context Protocol) Integration

WooCommerce now includes **native MCP support** (developer preview) — this is the flagship AI feature.

What MCP enables: AI assistants (Claude, Cursor, Windsurf, etc.) can directly interact with a WooCommerce store via a standardized protocol. This means an AI agent can query/create/update/delete products and orders through structured tools.

Architecture:



Available MCP Tools: - **Product Management:** Query (filter + paginate), Create, Update, Delete - **Order Management:** Query (filter + paginate), Update status, Add order notes - **REST-derived compatibility:** Product list/retrieve/create/update/delete, Order list/retrieve/create/update

Enabling MCP:

```
add_filter('woocommerce_features', function($features) {
    $features['mcp_integration'] = true;
    return $features;
});
```

Or via CLI: `wp option update woocommerce_feature_mcp_integration_enabled yes`

Claude Code integration:

```
claude mcp add woocommerce_mcp \  
--env WP_API_URL=https://yourstore.com/wp-json/woocommerce/mcp \  
--env CUSTOM_HEADERS='{\"X-MCP-API-Key\": \"YOUR_CONSUMER_KEY:YOUR_CONSUMER_SECRET\"}' \  
-- npx -y @automattic/mcp-wordpress-remote@latest
```

Key repos: - WordPress Abilities API: github.com/WordPress/abilities-api - WordPress MCP Adapter: github.com/WordPress/mcp-adapter - MCP WordPress Remote proxy: github.com/Automattic/mcp-wordpress-remote - Demo plugin: github.com/woocommerce/wc-mcp-ability

2.2 LLMS.txt — Feed Docs into AI IDEs

WooCommerce developer docs are AI-ingestible: - **llms.txt**: Table of contents with title/URL/description per doc - **llms-full.txt**: Full markdown export of ALL docs in one file - **.md suffix**: Append **.md** to any doc URL for markdown view - **Clipboard icon**: Copy any page as markdown for LLM paste

2.3 Agent Skills for AI Coding Assistants

The WooCommerce monorepo includes **.ai/skills/** directory with agent skills (SKILL.md files) for: - Backend PHP development and testing conventions - Code review standards - Git workflows and commit conventions - Build and linting processes - UI copy and documentation guidelines

Separate agent skills repos: - github.com/WordPress/agent-skills — foundational WordPress dev patterns - github.com/woocommerce/agent-skills — WooCommerce extension dev guidance

2.4 Cursor Rules

The **.cursor/rules/** directory in the monorepo provides AI assistant guidance for consistent WooCommerce development workflows.

3. WOOCOMMERCE MERCHANT AI FEATURES

3.1 AI Addons (by Plugify) — Official Marketplace Extension

URL: <https://woocommerce.com/products/ai-addons/>

A paid marketplace extension that brings AI into the WooCommerce admin:

Features: - **AI Chatbot:** 24/7 customer support chatbot powered by OpenAI - Configured under WooCommerce → Settings → AI Addons - Requires OpenAI API Key - Chat history viewable under WooCommerce > AI Chats - Can email customers directly from chat log - Page-selectable display (which pages show the chatbot) - **AI Product Description Generator:** "Generate Product Description" button appears below the product description editor — AI rewrites/improves it - **AI Image Generation:** Auto-generate product images - **AI Review Replies:** Auto-generate responses to customer reviews

3.2 SensAI (WooCommerce Extension)

Another AI extension on the Woo Marketplace — AI-powered store assistant. Details limited from scraped data.

3.3 AI Support Chat

WooCommerce.com now has live **AI Support Chat** on their documentation site — for customer support.

3.4 Avalara AI Tax Automation

Avalara's AI-powered tax compliance automation is integrated with WooCommerce for automatic tax calculation.

4. WORDPRESS.COM PLUGIN PAGE

URL: <https://wordpress.com/plugins/woocommerce>

Basic plugin listing. Key facts: - Version: 10.9.4 (Jul 7, 2026) - 7M+ active installations - 4.5 star rating - By Automattic - Free core, paid extensions model - No AI-specific content on the plugin listing page itself

5. WOOCOMMERCE OFFICIAL DOCUMENTATION

URL: <https://woocommerce.com/documentation/>

The docs portal is extensive. From scraping, I identified: - **AI Addons** has its own documentation section - **AutomateWoo** — workflow automation extension (marketing automation, not AI) - **SensAI** — AI store assistant extension - Documentation covers: WooCommerce core, WooPayments, Shipping, Tax, Subscriptions, Bookings, Memberships, and 50+ extensions - **Developer docs** are separate at developer.woocommerce.com/docs/

6. KEY ARCHITECTURAL INSIGHTS FOR AI-POWERED WOOCOMMERCE DEVELOPMENT

6.1 MCP is the Strategic AI Integration

The MCP integration is WooCommerce's **official AI strategy**. It's in developer preview but represents the future direction: - Not just a chatbot — full programmatic AI agent access to store operations - Works with any MCP-compatible AI client (Claude, Cursor, Windsurf, etc.) - Extensible via the WordPress Abilities API (third-party plugins can register custom abilities)

6.2 Dual API (Code-First GraphQL)

WooCommerce now has a **dual API** — write PHP classes, get auto-generated GraphQL. This is AI-friendly because GraphQL's typed schema is naturally machine-readable and IDE-friendly.

6.3 AI-Ready Documentation

The llms.txt / llms-full.txt system means WooCommerce docs are designed to be ingested by AI coding assistants. This is a deliberate strategy to make AI-powered WooCommerce development easier.

6.4 Agent Skills

The `.ai/skills/` directories signal WooCommerce is investing in AI-assisted development workflows. These skills are tool-agnostic (work with any AI coding assistant that supports the agent-skills format).

7. RECOMMENDED NEXT STEPS FOR JONBEATZ

1. **Enable MCP on a test store:** Try the MCP integration with Claude Code or Cursor
2. **Load WooCommerce docs into Cursor:** Add `llms.txt` or `llms-full.txt` as custom documentation
3. **Clone agent-skills repos:** `git clone https://github.com/woocommerce/agent-skills` into your project
4. **Explore AI Addons:** If budget allows, test the AI Addons extension for product description generation and chatbot
5. **Build custom MCP abilities:** Use the `wc-mcp-ability` demo plugin as a template for custom AI agent tools
6. **Check out SensAI:** Another AI extension worth evaluating